Java 并发编程面试题

大家好，我是小林。

Java 并发编程是面试里的重头戏，尤其是中高级岗位，这块几乎是必考的。很多人平时工作里用的多线程场景不多，或者用了但没深究原理，到了面试就容易被问住。这块知识点比较分散，从底层的内存模型到上层的线程池配置，跨度很大，但只要理清楚主线，其实是很有逻辑的。

这篇文章整理了 Java 并发面试中最常被问到的知识点，涵盖多线程基础、并发安全机制、锁原理、线程池，以及一些常见的并发场景题。内容偏向原理理解，不是让你背 API，而是帮你搞清楚"这东西为什么这么设计"。

有几块是面试里的高频区域，建议重点投入：

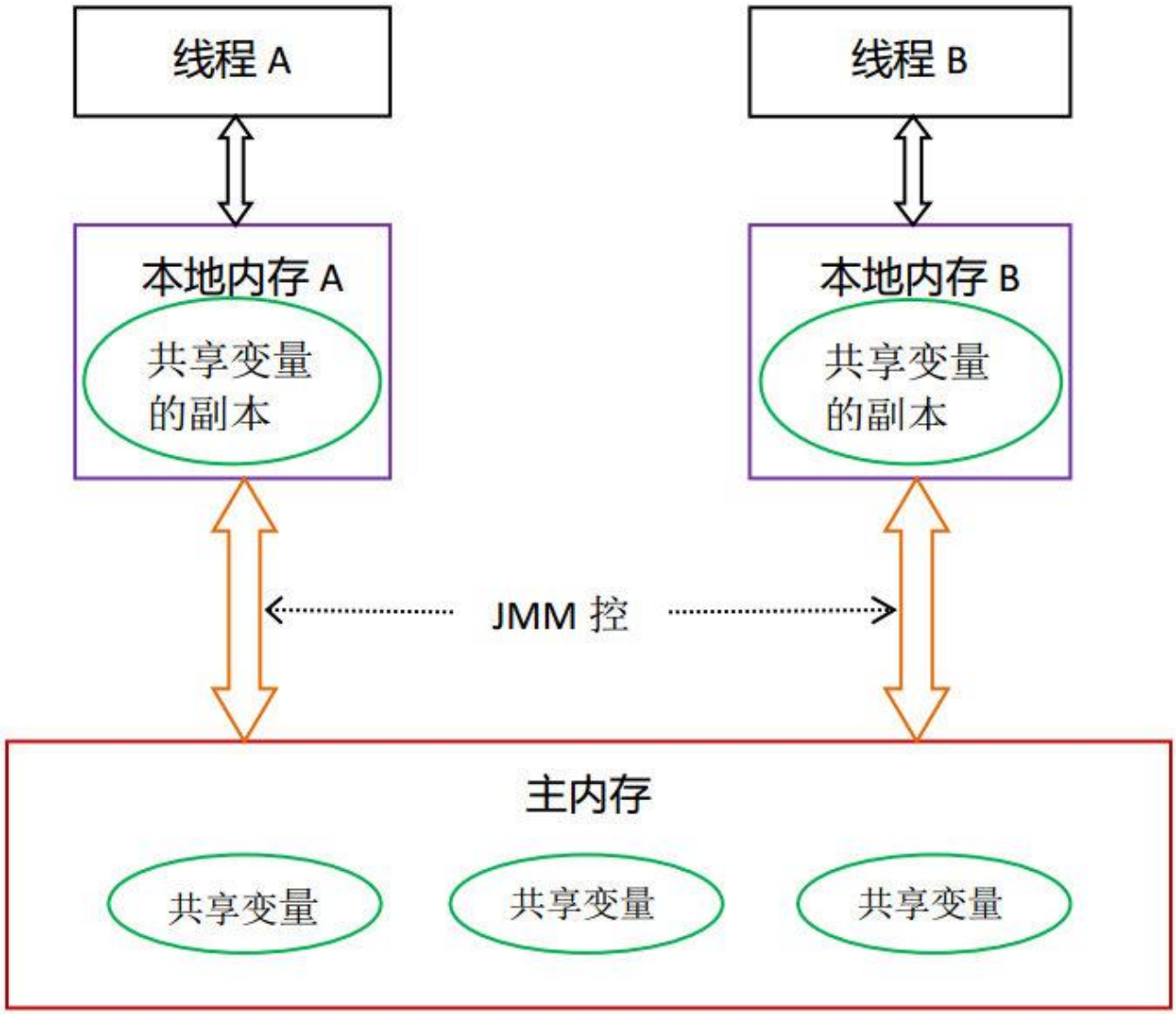
- **synchronized 和 ReentrantLock**：两者的区别、锁升级的过程（偏向锁 → 轻量级锁 → 重量级锁）、以及 AQS 是什么、怎么实现的，这条线是并发面试的核心主线。
- **volatile 和 CAS**：volatile 解决了什么问题、为什么不能保证原子性、CAS 的原理和 ABA 问题，这些问题入门好答，但深挖起来很多人答不全。
- **ThreadLocal**：用途、底层 ThreadLocalMap 的实现、为什么会内存泄漏、怎么避免，这个在面试里出现频率越来越高。
- **线程池**：核心参数的含义、任务提交后的执行流程、几种拒绝策略的区别、以及常见的线程池类型，这块是工程实践和面试的双重重点。
- **死锁**：死锁的四个必要条件、怎么排查、怎么避免，这是个老题，但每次还是会考。

如果你是第一次系统准备这块，建议先搞清楚 JMM 和 synchronized 的原理，再去看 AQS 和线程池，这样整体脉络会更清晰，后面的内容理解起来也会快很多。

多线程

Java 的内存模型（JMM）介绍一下

JMM 是专门解决多线程并发问题的一套规则。简单说，就是规定了多线程环境下，线程怎么访问共享变量才能不出错，核心是处理可见性、原子性、有序性这三个问题。



先想个场景：两个线程同时操作一个变量，比如线程 A 改了变量的值，线程 B 能不能立刻看到？

如果线程 B 看到的还是旧值，这就是可见性问题。为啥会这样？因为现在 CPU 都有缓存，线程操作变量时，会先把主内存里的变量读到自己的工作内存（比如 CPU 缓存）里，改完可能没及时写回主内存，另一个线程读的还是主内存的旧值。JMM 里像 volatile 关键字就专门解决这个，用了 volatile 的变量，改完会立刻刷回主内存，同时让其他线程的缓存失效，必须重新从主内存读，这样就保证了可见性。

再说说原子性，比如 i++，看着简单，其实是 “读 i、加 1、写回 i” 三步。如果两个线程同时做，可能线程 A 刚读完 i=10，线程 B 就把 i 改成 11，线程 A 再加 1 写回去还是 11，结果就错了。JMM 里用 synchronized 或者 Lock 锁就能保证原子性，加了锁之后，同一时间只有一个线程能执行这三步，中间不会被打断。

还有有序性，就是代码执行顺序可能和你写的不一样。编译器或者 CPU 为了提速，会在不影响单线程结果的情况下调整指令顺序。比如你先初始化对象 A，再把 A 的引用给变量 a，可能被重排成先给引用再初始化。单线程没问题，但多线程下，另一个线程拿到 a 的引用时，A 可能还没初始化

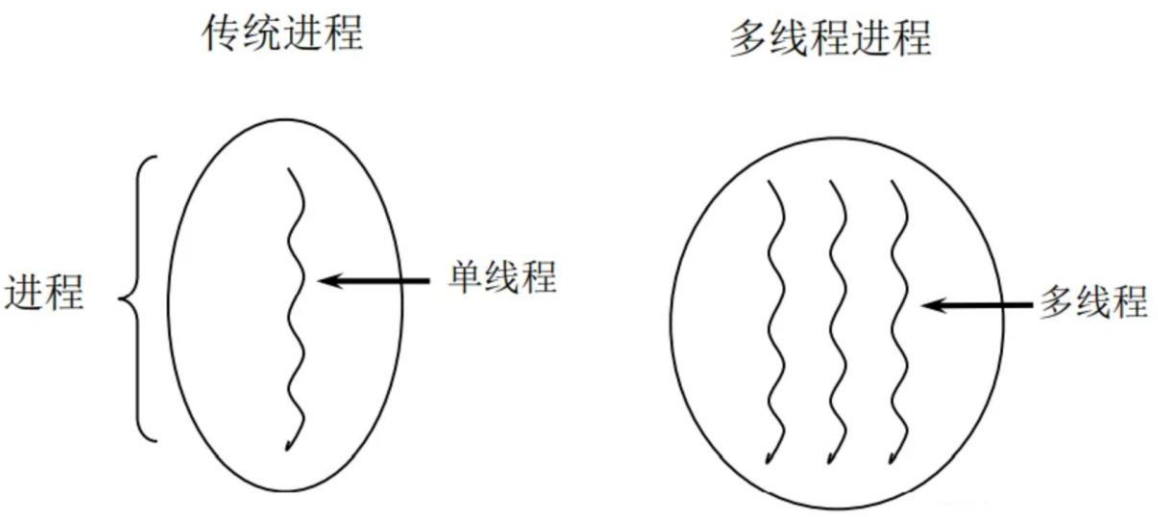
好，一调用就报错。这时候 volatile 或者 synchronized 就能通过内存屏障阻止这种重排序，保证顺序正确。

所以 JMM 的核心思路是：定义主内存（大家共享的内存）和工作内存（每个线程自己的缓存），规定变量必须从主内存加载到工作内存才能操作，改完再写回主内存。

然后通过 volatile、synchronized 这些关键字，控制加载、写回的时机，以及禁止不合理的指令重排，最终保证多线程操作共享变量时能正确交互。理解这些，就能明白为啥写并发代码时要用这些关键字，以及它们背后的原理了。

java 多线程是什么？需要注意什么？

Java 多线程是指在一个 Java 程序中同时运行多个线程，这些线程共享程序的内存空间（如全局变量、方法区等），但有各自的栈和程序计数器，能同时执行不同的任务，比如一个线程处理用户输入，另一个线程后台下载文件，提升程序效率。

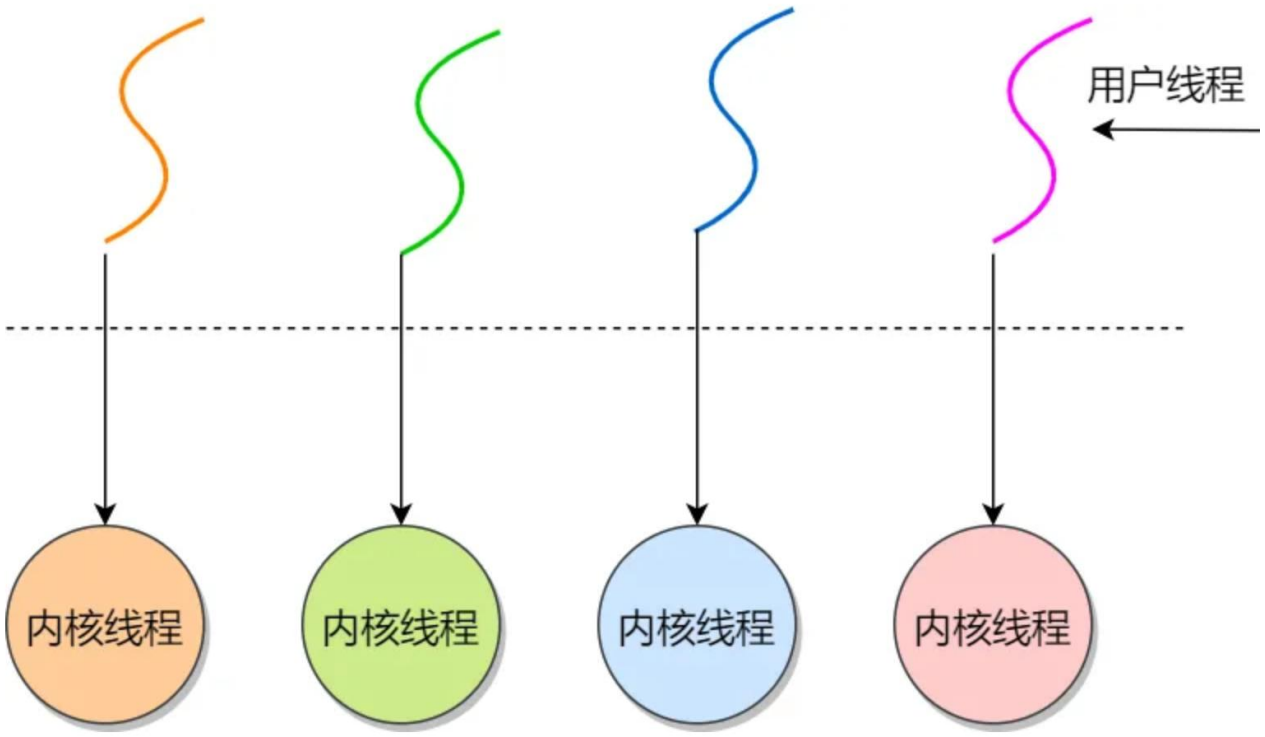


使用 Java 多线程需要注意以下几点：

- 首先是线程安全问题。多个线程同时操作共享数据时，可能出现错误。比如两个线程同时给一个变量加 1，原本该加 2，结果可能只加了 1，这是因为线程切换时没做好数据保护。需要用 `synchronized` 关键字、`Lock` 锁等方式，保证同一时间只有一个线程操作共享数据。
- 其次是线程间通信。线程需要协作时，比如一个线程生产数据，另一个线程消费数据，要通过 `wait()`、`notify()`等方法控制，避免出现一方没准备好，另一方就操作的情况，否则可能导致数据错误或线程无限等待。
- 然后是线程的创建和销毁成本。频繁创建和销毁线程会消耗系统资源，影响性能。可以用线程池管理线程，提前创建好一定数量的线程，重复使用，减少资源消耗。

java 里面的线程和操作系统的线程一样吗?

Java 底层会调用 pthread_create 来创建线程，所以本质上 java 程序创建的线程，就是和操作系统线程是一样的，是 1 对 1 的线程模型。



使用多线程要注意哪些问题?

要保证多线程的程序是安全，不要出现数据竞争造成的数据混乱的问题。

Java 的线程安全在三个方面体现：

- **原子性**：提供互斥访问，同一时刻只能有一个线程对数据进行操作，在 Java 中使用了 atomic 包（这个包提供了一些支持原子操作的类，这些类可以在多线程环境下保证操作的原子性）和 synchronized 关键字来确保原子性；
- **可见性**：一个线程对主内存的修改可以及时地被其他线程看到，在 Java 中使用了 synchronized 和 volatile 这两个关键字确保可见性；
- **有序性**：一个线程观察其他线程中的指令执行顺序，由于指令重排序，该观察结果一般杂乱无序，在 Java 中使用了 happens-before 原则来确保有序性。

保证数据的一致性有哪些方案呢?

- **事务管理**：使用数据库事务来确保一组数据库操作要么全部成功提交，要么全部失败回滚。通过 ACID（原子性、一致性、隔离性、持久性）属性，数据库事务可以保证数据的一致性。

- **锁机制**：使用锁来实现对共享资源的互斥访问。在 Java 中，可以使用 synchronized 关键字、ReentrantLock 或其他锁机制来控制并发访问，从而避免并发操作导致数据不一致。
- **版本控制**：通过乐观锁的方式，在更新数据时记录数据的版本信息，从而避免同时对同一数据进行修改，进而保证数据的一致性。

线程的创建方式有哪些?

> 1.继承 Thread 类

这是最直接的一种方式，用户自定义类继承 java.lang.Thread 类，重写其 run()方法，run()方法中定义了线程执行的具体任务。创建该类的实例后，通过调用 start()方法启动线程。

```
class MyThread extends Thread {
    @Override
    public void run() {
        // 线程执行的代码
    }
}

public static void main(String[] args) {
    MyThread t = new MyThread();
    t.start();
}
```

采用继承 Thread 类方式

- 优点: 编写简单，如果需要访问当前线程，无需使用 Thread.currentThread ()方法，直接使用 this，即可获得当前线程
- 缺点:因为线程类已经继承了 Thread 类，所以不能再继承其他的父类

> 2.实现 Runnable 接口

如果一个类已经继承了其他类，就不能再继承 Thread 类，此时可以实现 java.lang.Runnable 接口。实现 Runnable 接口需要重写 run()方法，然后将此 Runnable 对象作为参数传递给 Thread 类的构造器，创建 Thread 对象后调用其 start()方法启动线程。

```
class MyRunnable implements Runnable {
    @Override
```

```
    public void run() {
        // 线程执行的代码
    }
}

public static void main(String[] args) {
    Thread t = new Thread(new MyRunnable());
    t.start();
}
```

采用实现 Runnable 接口方式：

- 优点：线程类只是实现了 Runable 接口，还可以继承其他的类。在这种方式下，可以多个线程共享同一个目标对象，所以非常适合多个相同线程来处理同一份资源的情况，从而可以将 CPU 代码和数据分开，形成清晰的模型，较好地体现了面向对象的思想。
- 缺点：编程稍微复杂，如果需要访问当前线程，必须使用 Thread.currentThread()方法。

> 3. 实现 Callable 接口与 FutureTask

java.util.concurrent.Callable 接口类似于 Runnable，但 Callable 的 call()方法可以有返回值并且可以抛出异常。要执行 Callable 任务，需将它包装进一个 FutureTask，因为 Thread 类的构造器只接受 Runnable 参数，而 FutureTask 实现了 Runnable 接口。

```
class MyCallable implements Callable<Integer> {
    @Override
    public Integer call() throws Exception {
        // 线程执行的代码，这里返回一个整型结果
        return 1;
    }
}

public static void main(String[] args) {
    MyCallable task = new MyCallable();
    FutureTask<Integer> futureTask = new FutureTask<>(task);
    Thread t = new Thread(futureTask);
    t.start();
```

```
    try {
        Integer result = futureTask.get();  // 获取线程执行结果
        System.out.println("Result: " + result);
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();
    }
}
```

采用实现 Callable 接口方式：

- 缺点：编程稍微复杂，如果需要访问当前线程，必须调用 Thread.currentThread()方法。
- 优点：线程只是实现 Runnable 或实现 Callable 接口，还可以继承其他类。这种方式下，多个线程可以共享一个 target 对象，非常适合多线程处理同一份资源的情形。

> 4. 使用线程池（Executor 框架）

从 Java 5 开始引入的 java.util.concurrent.ExecutorService 和相关类提供了线程池的支持，这是一种更高效的线程管理方式，避免了频繁创建和销毁线程的开销。可以通过 Executors 类的静态方法创建不同类型的线程池。

```
class Task implements Runnable {
    @Override
    public void run() {
        // 线程执行的代码
    }
}

public static void main(String[] args) {
    ExecutorService executor = Executors.newFixedThreadPool(10);  // 创建固定大小的线程池
    for (int i = 0; i < 100; i++) {
        executor.submit(new Task());  // 提交任务到线程池执行
    }
    executor.shutdown();  // 关闭线程池
}
```

采用线程池方式：

- 缺点：程池增加了程序的复杂度，特别是当涉及线程池参数调整和故障排查时。错误的配置可能导致死锁、资源耗尽等问题，这些问题的诊断和修复可能较为复杂。
- 优点：线程池可以重用预先创建的线程，避免了线程创建和销毁的开销，显著提高了程序的性能。对于需要快速响应的并发请求，线程池可以迅速提供线程来处理任务，减少等待时间。并且，线程池能够有效控制运行的线程数量，防止因创建过多线程导致的系统资源耗尽（如内存溢出）。通过合理配置线程池大小，可以最大化 CPU 利用率和系统吞吐量。

怎么启动线程？

启动线程的通过 Thread 类的 `start()`。

```
//创建两个线程，用 start 启动线程
MyThread myThread1 = new MyThread();
MyThread myThread2 = new MyThread();
myThread1.start();
myThread2.start();
```

如何停止一个线程的运行?

主要有这些方法：

- **异常法停止**：线程调用 interrupt()方法后，在线程的 run 方法中判断当前对象的 interrupted()状态，如果是中断状态则抛出异常，达到中断线程的效果。
- **在沉睡中停止**：先将线程 sleep，然后调用 interrupt 标记中断状态，interrupt 会将阻塞状态的线程中断。会抛出中断异常，达到停止线程的效果
- **stop()暴力停止**：线程调用 stop()方法会被暴力停止，方法已弃用，该方法会有不好的后果：强制让线程停止有可能使一些请理性的工作得不到完成。
- **使用 return 停止线程**：调用 interrupt 标记为中断状态后，在 run 方法中判断当前线程状态，如果为中断状态则 return，能达到停止线程的效果。

调用 interrupt 是如何让线程抛出异常的?

每个线程都一个与之关联的布尔属性来表示其中断状态，中断状态的初始值为 false，当一个线程被其它线程调用 `Thread.interrupt()`方法中断时，会根据实际情况做出响应。

- 如果该线程正在执行低级别的可中断方法（如 `Thread.sleep()`、`Thread.join()`或 `Object.wait()`），则会解除阻塞并**抛出 `InterruptedException` 异常**。

- 否则 `Thread.interrupt()`仅设置线程的中断状态，在该被中断的线程中稍后可通过轮询中断状态来决定是否要停止当前正在执行的任务。

Java 线程的状态有哪些?

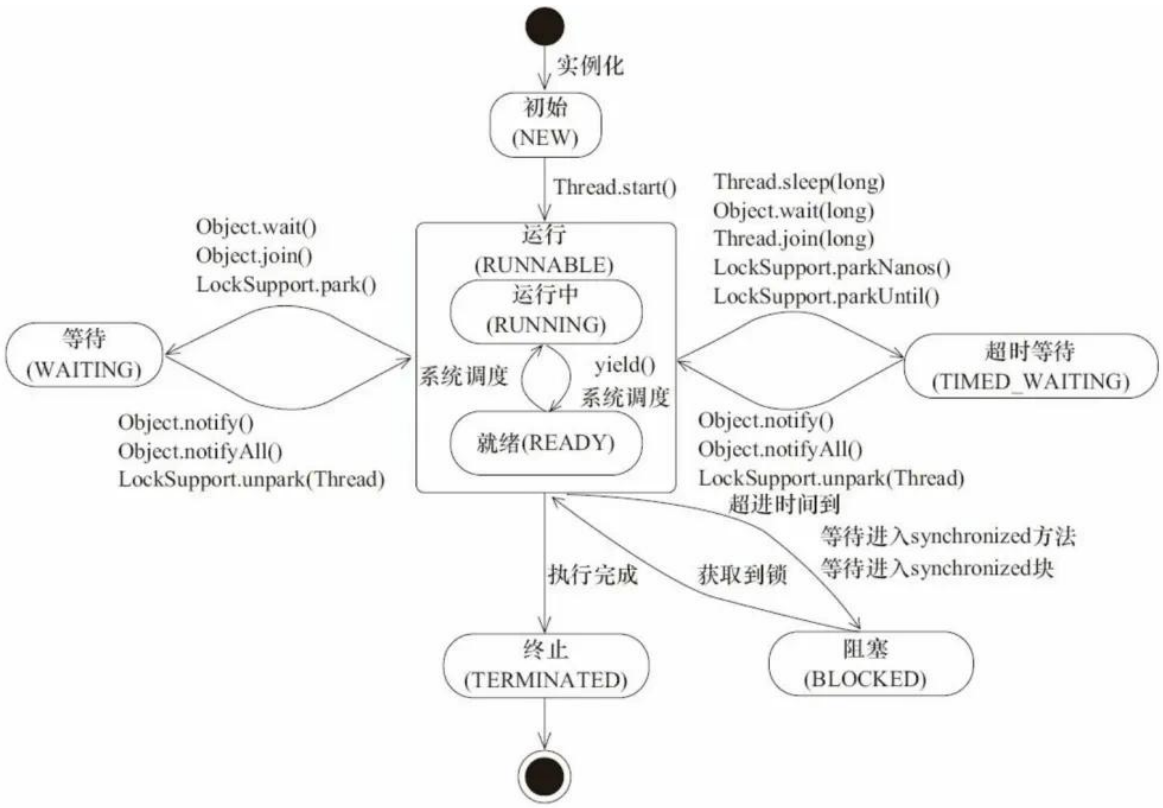


图4-1 Java线程状态变迁

源自《Java 并发编程艺术》 java.lang.Thread.State 枚举类中定义了六种线程的状态，可以调用线程 Thread 中的 getState()方法**获取当前线程的状态**。

线程状态	解释
NEW	尚未启动的线程状态，即线程创建，**还未调用 start 方法**
RUNNABLE	**就绪状态**（调用 start，等待调度）+**正在运行**
BLOCKED	**等待监视器锁**时，陷入阻塞状态

线程状态	解释
WAITING	等待状态的线程正在**等待**另一线程执行特定的操作（如 notify）
TIMED_WAITING	具有**指定等待时间**的等待状态
TERMINATED	线程完成执行，**终止状态**

sleep 和 wait 的区别是什么?

对比例表：

特性	`sleep()`	`wait()`
所属类	`Thread` 类（静态方法）	`Object` 类（实例方法）
锁释放	❌	☑
使用前提	任意位置调用	必须在同步块内（持有锁）
唤醒机制	超时自动恢复	需 `notify()`/`notifyAll()` 或超时
设计用途	暂停线程执行，不涉及锁协作	线程间协调，释放锁让其他线程工作

- **所属分类的不同**：sleep 是 `Thread` 类的静态方法，可以在任何地方直接通过 `Thread.sleep()` 调用，无需依赖对象实例。wait 是 `Object` 类的实例方法，这意味着必须通过对象实例来调用。
- **锁释放的情况**：`Thread.sleep()` 在调用时，线程会暂停执行指定的时间，但不会释放持有的对象锁。也就是说，在 `sleep` 期间，其他线程无法获得该线程持有的锁。`Object.wait()`：调

用该方法时，线程会释放持有的对象锁，进入等待状态，直到其他线程调用相同对象的 `notify()` 或 `notifyAll()` 方法唤醒它

- **使用条件**：sleep 可在任意位置调用，无需事先获取锁。 wait 必须在同步块或同步方法内调用（即线程需持有该对象的锁），否则抛出 `IllegalMonitorStateException`。
- **唤醒机制**：sleep 休眠时间结束后，线程 自动恢复 到就绪状态，等待 CPU 调度。wait 需要其他线程调用相同对象的 `notify()` 或 `notifyAll()` 方法才能被唤醒。`notify()` 会随机唤醒一个在该对象上等待的线程，而 `notifyAll()` 会唤醒所有在该对象上等待的线程。

sleep 会释放 cpu 吗?

是的，调用 `Thread.sleep()` 时，线程会释放 CPU，但不会释放持有的锁。

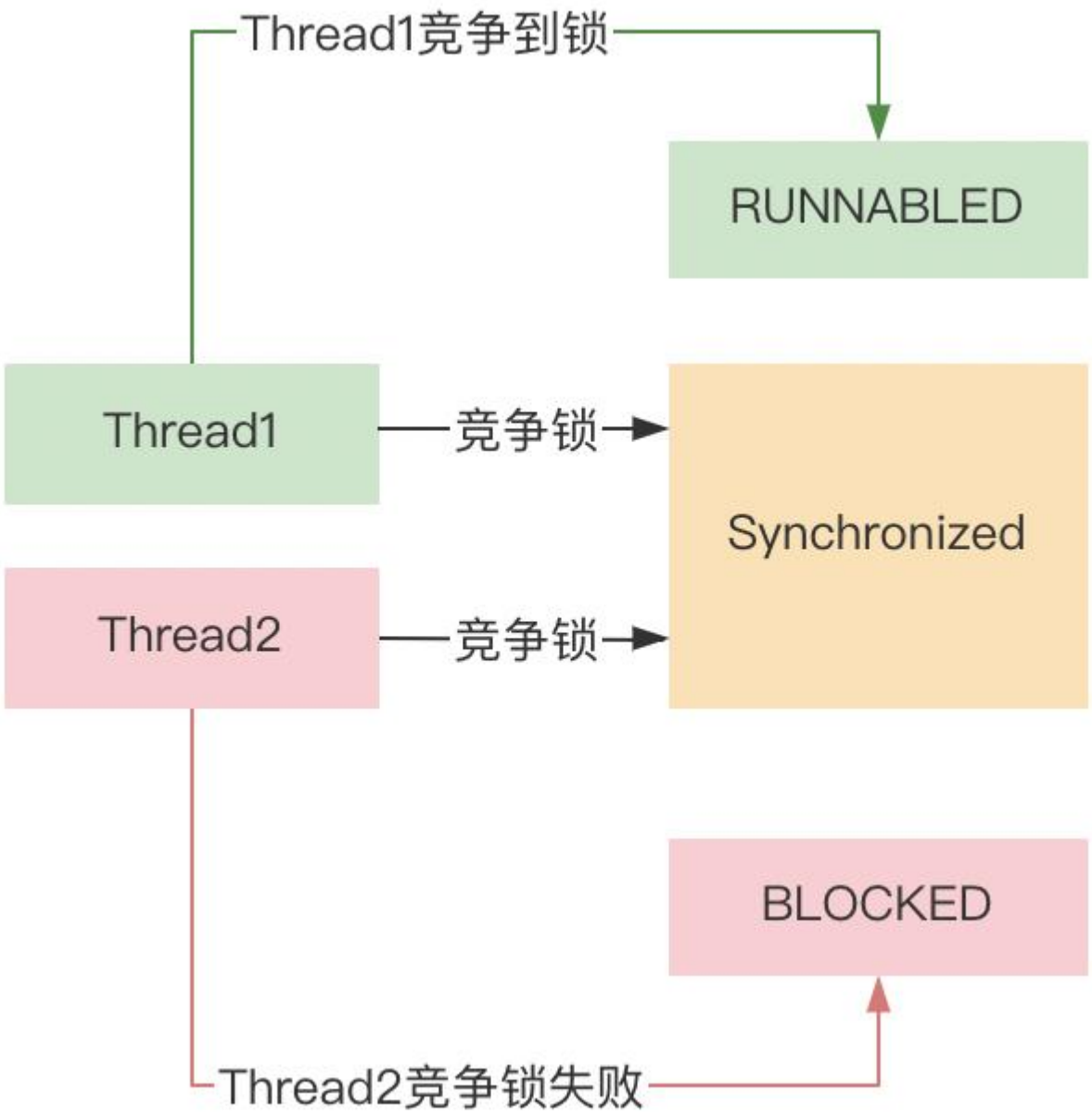
当线程调用 `sleep()` 后，会主动让出 CPU 时间片，进入 `TIMED_WAITING` 状态。此时操作系统会触发调度，将 CPU 分配给其他处于就绪状态的线程。这样其他线程（无论是需要同一锁的线程还是不相关线程）便有机会执行。

`sleep()` 不会释放线程已持有的任何锁（如 `synchronized` 同步代码块或方法中获取的锁）。因此，如果有其他线程试图获取同一把锁，它们仍会被阻塞，直到原线程退出同步代码块。

blocked 和 waiting 有啥区别

区别如下：

- **触发条件**:线程进入 BLOCKED 状态通常是因为试图获取一个对象的锁（monitor lock），但该锁已经被另一个线程持有。这通常发生在尝试进入 synchronized 块或方法时，如果锁已被占用，则线程将被阻塞直到锁可用。线程进入 WAITING 状态是因为它正在等待另一个线程执行某些操作，例如调用 Object.wait()方法、Thread.join()方法或 LockSupport.park()方法。在这种状态下，线程将不会消耗 CPU 资源，并且不会参与锁的竞争。



- **唤醒机制**:当一个线程被阻塞等待锁时，一旦锁被释放，线程将有机会重新尝试获取锁。如果锁此时未被其他线程获取，那么线程可以从 BLOCKED 状态变为 RUNNABLE 状态。线程在 WAITING 状态中需要被显式唤醒。例如，如果线程调用了 Object.wait()，那么它必须等待另一个线程调用同一对象上的 Object.notify()或 Object.notifyAll()方法才能被唤醒。

所以，BLOCKED 和 WAITING 两个状态最大的区别有两个：

- BLOCKED 是锁竞争失败后被被动触发的状态，WAITING 是人为的主动触发的状态
- BLCKED 的唤醒时自动触发的，而 WAITING 状态是必须要通过特定的方法来主动唤醒

wait 状态下的线程如何进行恢复到 running 状态?

线程从 `等待（WAIT）` 状态恢复到 `运行（RUNNING）` 状态的核心机制是 **通过外部事件触发或资源可用性变化**，比如等待的线程**被其他线程对象唤醒**，`notify()`和 `notifyAll()`。

```
synchronized (lock) {
    // 线程进入等待状态，释放锁
    lock.wait();
}

// 其他线程调用以下代码唤醒等待线程
synchronized (lock) {
    lock.notify();      // 唤醒单个线程
    // lock.notifyAll(); // 唤醒所有等待线程
}
```

notify 和 notifyAll 的区别?

同样是唤醒等待的线程，同样最多只有一个线程能获得锁，同样不能控制哪个线程获得锁。

区别在于：

- notify：唤醒一个线程，其他线程依然处于 wait 的等待唤醒状态，如果被唤醒的线程结束时没调用 notify，其他线程就永远没人去唤醒，只能等待超时，或者被中断
- notifyAll：所有线程退出 wait 的状态，开始竞争锁，但只有一个线程能抢到，这个线程执行完后，其他线程又会有一个幸运儿脱颖而出得到锁

notify 选择哪个线程?

notify 在源码的注释中说到 notify 选择唤醒的线程是任意的，但是依赖于具体实现的 jvm。

```
/**
 * Wakes up a single thread that is waiting on this object's
 * monitor. If any threads are waiting on this object, one of them
 * is chosen to be awakened. The choice is arbitrary and occurs at
 * the discretion of the implementation. A thread waits on an object's
 * monitor by calling one of the {@code wait} methods.
 * <p>
```

JVM 有很多实现，比较流行的就是 hotspot，hotspot 对 notofy()的实现并不是我们以为的随机唤醒,，而是“先进先出”的顺序唤醒。

不同的线程之间如何通信?

共享变量是最基本的线程间通信方式。多个线程可以访问和修改同一个共享变量，从而实现信息的传递。为了保证线程安全，通常需要使用 `synchronized` 关键字或 `volatile` 关键字。

```
class SharedVariableExample {
    // 使用 volatile 关键字保证变量的可见性
    private static volatile boolean flag = false;

    public static void main(String[] args) {
        // 生产者线程
        Thread producer = new Thread(() -> {
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            // 修改共享变量
            flag = true;
            System.out.println("Producer: Flag is set to true.");
        });

        // 消费者线程
        Thread consumer = new Thread(() -> {
            while (!flag) {
                // 等待共享变量被修改
            }
            System.out.println("Consumer: Flag is now true.");
        });

        producer.start();
        consumer.start();
    }
}
```

代码解释

- `volatile` 关键字确保了 `flag` 变量在多个线程之间的可见性，即一个线程修改了 `flag` 的值，其他线程能立即看到。
- 生产者线程在睡眠 2 秒后将 `flag` 设置为 `true`，消费者线程在 `flag` 为 `false` 时一直等待，直到 `flag` 变为 `true` 才继续执行。

`Object` 类中的 `wait()`、`notify()` 和 `notifyAll()` 方法可以用于线程间的协作。`wait()` 方法使当前线程进入等待状态，`notify()` 方法唤醒在此对象监视器上等待的单个线程，`notifyAll()` 方法唤醒在此对象监视器上等待的所有线程。

```
class WaitNotifyExample {
    private static final Object lock = new Object();

    public static void main(String[] args) {
        // 生产者线程
        Thread producer = new Thread(() -> {
            synchronized (lock) {
                try {
                    System.out.println("Producer: Producing...");
                    Thread.sleep(2000);
                    System.out.println("Producer: Production finished. Notifying consumer.");
                    // 唤醒等待的线程
                    lock.notify();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });

        // 消费者线程
        Thread consumer = new Thread(() -> {
            synchronized (lock) {
                try {
                    System.out.println("Consumer: Waiting for production to finish.");
                    // 进入等待状态
                    lock.wait();
```

```
                    System.out.println("Consumer: Production finished. Consuming...");
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });

        consumer.start();
        producer.start();
    }
}
```

代码解释：

- `lock` 是一个用于同步的对象，生产者和消费者线程都需要获取该对象的锁才能执行相应的操作。
- 消费者线程调用 `lock.wait()` 方法进入等待状态，释放锁；生产者线程执行完生产任务后调用 `lock.notify()` 方法唤醒等待的消费者线程。

`java.util.concurrent.locks` 包中的 `Lock` 和 `Condition` 接口提供了比 `synchronized` 更灵活的线程间通信方式。`Condition` 接口的 `await()` 方法类似于 `wait()` 方法，`signal()` 方法类似于 `notify()` 方法，`signalAll()` 方法类似于 `notifyAll()` 方法。

```
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class LockConditionExample {
    private static final Lock lock = new ReentrantLock();
    private static final Condition condition = lock.newCondition();

    public static void main(String[] args) {
        // 生产者线程
        Thread producer = new Thread(() -> {
            lock.lock();
            try {
                System.out.println("Producer: Producing...");
```

```
                Thread.sleep(2000);
                System.out.println("Producer: Production finished. Notifying consumer.");
                // 唤醒等待的线程
                condition.signal();
            } catch (InterruptedException e) {
                e.printStackTrace();
            } finally {
                lock.unlock();
            }
        });

        // 消费者线程
        Thread consumer = new Thread(() -> {
            lock.lock();
            try {
                System.out.println("Consumer: Waiting for production to finish.");
                // 进入等待状态
                condition.await();
                System.out.println("Consumer: Production finished. Consuming...");
            } catch (InterruptedException e) {
                e.printStackTrace();
            } finally {
                lock.unlock();
            }
        });

        consumer.start();
        producer.start();
    }
}
```

代码解释：

- `ReentrantLock` 是 `Lock` 接口的一个实现类，`condition` 是通过 `lock.newCondition()` 方法创建的。

- 消费者线程调用 `condition.await()` 方法进入等待状态，生产者线程执行完生产任务后调用 `condition.signal()` 方法唤醒等待的消费者线程。

`java.util.concurrent` 包中的 `BlockingQueue` 接口提供了线程安全的队列操作，当队列满时，插入元素的线程会被阻塞；当队列为空时，获取元素的线程会被阻塞。

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

class BlockingQueueExample {
    private static final BlockingQueue<Integer> queue = new LinkedBlockingQueue<>(1);

    public static void main(String[] args) {
        // 生产者线程
        Thread producer = new Thread(() -> {
            try {
                System.out.println("Producer: Producing...");
                queue.put(1);
                System.out.println("Producer: Production finished.");
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        // 消费者线程
        Thread consumer = new Thread(() -> {
            try {
                System.out.println("Consumer: Waiting for production to finish.");
                int item = queue.take();
                System.out.println("Consumer: Consumed item: " + item);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        consumer.start();
```

```
        producer.start();
    }
}
```

代码解释：

- `LinkedBlockingQueue` 是 `BlockingQueue` 接口的一个实现类，容量为 1。
- 生产者线程调用 `queue.put(1)` 方法将元素插入队列，如果队列已满，线程会被阻塞；消费者线程调用 `queue.take()` 方法从队列中取出元素，如果队列为空，线程会被阻塞。

线程间通信方式有哪些?

1、Object 类的 wait()、notify() 和 notifyAll() 方法。这是 Java 中最基础的线程间通信方式，基于对象的监视器（锁）机制。

- `wait()`：使当前线程进入等待状态，直到其他线程调用该对象的 `notify()` 或 `notifyAll()` 方法。
- `notify()`：唤醒在此对象监视器上等待的单个线程。
- `notifyAll()`：唤醒在此对象监视器上等待的所有线程。

```
class SharedObject {
    public synchronized void consumerMethod() throws InterruptedException {
        while (/* 条件不满足 */) {
            wait();
        }
        // 执行相应操作
    }

    public synchronized void producerMethod() {
        // 执行相应操作
        notify(); // 或者 notifyAll()
    }
}
```

2、`Lock` 和 `Condition` 接口。`Lock` 接口提供了比 `synchronized` 更灵活的锁机制，`Condition` 接口则配合 `Lock` 实现线程间的等待 / 通知机制。

- `await()`：使当前线程进入等待状态，直到被其他线程唤醒。

- `signal()`：唤醒一个等待在该 `Condition` 上的线程。
- `signalAll()`：唤醒所有等待在该 `Condition` 上的线程。

```
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class SharedResource {
    private final Lock lock = new ReentrantLock();
    private final Condition condition = lock.newCondition();

    public void consumer() throws InterruptedException {
        lock.lock();
        try {
            while (/* 条件不满足 */) {
                condition.await();
            }
            // 执行相应操作
        } finally {
            lock.unlock();
        }
    }

    public void producer() {
        lock.lock();
        try {
            // 执行相应操作
            condition.signal(); // 或者 signalAll()
        } finally {
            lock.unlock();
        }
    }
}
```

3、`volatile` 关键字。`volatile` 关键字用于保证变量的可见性，即当一个变量被声明为 `volatile` 时，它会保证对该变量的写操作会立即刷新到主内存中，而读操作会从主内存中读取最新的值。

```
class VolatileExample {
    private volatile boolean flag = false;

    public void writer() {
        flag = true;
    }

    public void reader() {
        while (!flag) {
            // 等待
        }
        // 执行相应操作
    }
}
```

4、CountDownLatch。`CountDownLatch` 是一个同步辅助类，它允许一个或多个线程等待其他线程完成操作。

- `CountDownLatch(int count)`：构造函数，指定需要等待的线程数量。
- `countDown()`：减少计数器的值。
- `await()`：使当前线程等待，直到计数器的值为 0。

```
import java.util.concurrent.CountDownLatch;

public class CountDownLatchExample {
    public static void main(String[] args) throws InterruptedException {
        int threadCount = 3;
        CountDownLatch latch = new CountDownLatch(threadCount);

        for (int i = 0; i < threadCount; i++) {
            new Thread(() -> {
                try {
                    // 执行任务
```

```
                    System.out.println(Thread.currentThread().getName() + " 完成任务");
                } finally {
                    latch.countDown();
                }
            }).start();
        }

        latch.await();
        System.out.println("所有线程任务完成");
    }
}
```

5、CyclicBarrier。`CyclicBarrier` 是一个同步辅助类，它允许一组线程相互等待，直到所有线程都到达某个公共屏障点。

- `CyclicBarrier(int parties, Runnable barrierAction)`：构造函数，指定参与的线程数量和所有线程到达屏障点后要执行的操作。
- `await()`：使当前线程等待，直到所有线程都到达屏障点。

```
import java.util.concurrent.CyclicBarrier;

public class CyclicBarrierExample {
    public static void main(String[] args) {
        int threadCount = 3;
        CyclicBarrier barrier = new CyclicBarrier(threadCount, () -> {
            System.out.println("所有线程都到达屏障点");
        });

        for (int i = 0; i < threadCount; i++) {
            new Thread(() -> {
                try {
                    // 执行任务
                    System.out.println(Thread.currentThread().getName() + " 到达屏障点");
                    barrier.await();
                    // 继续执行后续任务
                } catch (Exception e) {
```

```
                    e.printStackTrace();
                }
            }).start();
        }
    }
}
```

6、Semaphore。`Semaphore` 是一个计数信号量，它可以控制同时访问特定资源的线程数量。

- `Semaphore(int permits)`：构造函数，指定信号量的初始许可数量。
- `acquire()`：获取一个许可，如果没有可用许可则阻塞。
- `release()`：释放一个许可。

```
import java.util.concurrent.Semaphore;

public class SemaphoreExample {
    public static void main(String[] args) {
        int permitCount = 2;
        Semaphore semaphore = new Semaphore(permitCount);

        for (int i = 0; i < 5; i++) {
            new Thread(() -> {
                try {
                    semaphore.acquire();
                    System.out.println(Thread.currentThread().getName() + " 获得许可");
                    // 执行任务
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                } finally {
                    semaphore.release();
                    System.out.println(Thread.currentThread().getName() + " 释放许可");
                }
            }).start();
        }
    }
```

```
}
```

如何停止一个线程?

在 Java 中，停止线程的正确方式是 **通过协作式的逻辑控制线程终止**，而非强制暴力终止（如已废弃的 `Thread.stop()`）。以下是实现安全停止线程的多种方法：

第一种方式：通过共享标志位主动终止。定义一个 可见的 状态变量，由主线程控制其值，工作线程循环检测该变量以决定是否退出。

```
public class SafeStopWithFlag implements Runnable {
    // 使用 volatile 保证可见性
    private volatile boolean running = true;

    @Override
    public void run() {
        while (running) {
            try {
                // 处理任务逻辑
                System.out.println("Thread is running...");
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                // 捕获中断异常后设置 running=false
                running = false;
                Thread.currentThread().interrupt(); // 重新设置中断标志
            }
        }
        System.out.println("Thread terminated safely.");
    }

    // 停止线程的方法（由外部调用）
    public void stop() {
        running = false;
    }
}
```

调用方式：

```
SafeStopWithFlag task = new SafeStopWithFlag();
Thread thread = new Thread(task);
thread.start();
// 某个时刻调用停止
Thread.sleep(3000);
task.stop();
```

第二种方式使用线程中断机制。通过 `Thread.interrupt()` 触发线程中断状态，结合中断检测逻辑实现安全停止。

```
public class InterruptExample implements Runnable {
    @Override
    public void run() {
        while (!Thread.currentThread().isInterrupted()) {
            try {
                System.out.println("Working...");
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                // 当阻塞时被中断，抛出异常并清除中断状态
                System.out.println("Interrupted during sleep!");
                Thread.currentThread().interrupt(); // 重新设置中断标志
            }
        }
        System.out.println("Thread terminated by interrupt.");
    }
}
```

调用方式：

```
Thread thread = new Thread(new InterruptExample());
thread.start();
// 中断线程
Thread.sleep(3000);
thread.interrupt();
```

- `interrupt()` 不会立刻终止线程，只是设置中断标志位。

- 线程需手动检查中断状态（`isInterrupted()`）或触发可中断操作（如 `sleep()`，`wait()`，`join()`）响应中断。
- 阻塞操作中收到中断请求时，会抛出 `InterruptedException` 并清除中断状态。

第三种方式通过 `Future` 取消任务。使用线程池提交任务，并通过 `Future.cancel()` 停止线程，依赖中断机制。

```
public class FutureCancelDemo {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newSingleThreadExecutor();
        Future<?> future = executor.submit(() -> {
            while (!Thread.currentThread().isInterrupted()) {
                System.out.println("Task running...");
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    System.out.println("Task interrupted.");
                    Thread.currentThread().interrupt();
                }
            }
        });

        try {
            Thread.sleep(3000);
            future.cancel(true); // true 表示尝试中断任务线程
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            executor.shutdown();
        }
    }
}
```

第四种方式处理不可中断的阻塞操作。某些 I/O 或同步操作（如 `Socket.accept()`、`Lock.lock()`）无法通过中断直接响应。此时需结合资源关闭操作。比如，关闭 Socket 释放阻塞。

```
public class SocketHandler implements Runnable {
    private ServerSocket serverSocket;

    public SocketHandler(ServerSocket serverSocket) {
        this.serverSocket = serverSocket;
    }

    @Override
    public void run() {
        try {
            // serverSocket.accept()阻塞时无法响应中断
            while (!Thread.currentThread().isInterrupted()) {
                Socket socket = serverSocket.accept();
                // 处理连接...
            }
        } catch (IOException e) {
            if (Thread.currentThread().isInterrupted()) {
                System.out.println("Thread stopped by interrupt.");
            }
        }
    }

    // 特殊关闭方法（销毁资源）
    public void stop() {
        try {
            serverSocket.close(); // 关闭资源使 accept()抛出异常
        } catch (IOException e) {
            System.out.println("Error closing socket: " + e);
        }
    }
}
```

调用方式：调用 `stop()` 方法关闭资源以解除阻塞。

线程停止的正确实践，如下表格：

方法	适用场景	注意事项
循环检测标志位	简单无阻塞的逻辑	确保标志位使用 `volatile` 或通过锁保证可见性
中断机制	可中断的阻塞操作	正确处理 `InterruptedException` 并恢复中断标志
`Future.cancel()`	线程池管理任务	需要线程池任务支持中断处理机制
资源关闭	不可中断的阻塞操作（如 Sockets）	显式关闭资源触发异常，结合中断状态判断回滚

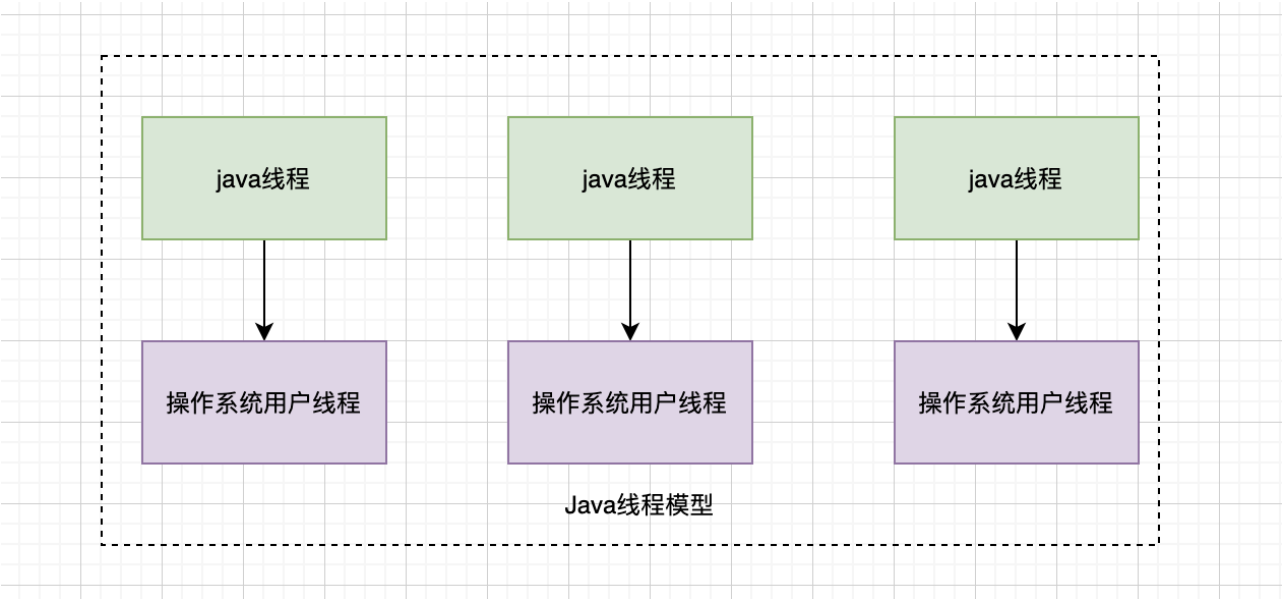
避免使用以下已废弃方法：

- `Thread.stop()`：暴力终止，可能导致状态不一致。
- `Thread.suspend()/resume()`：易导致死锁。

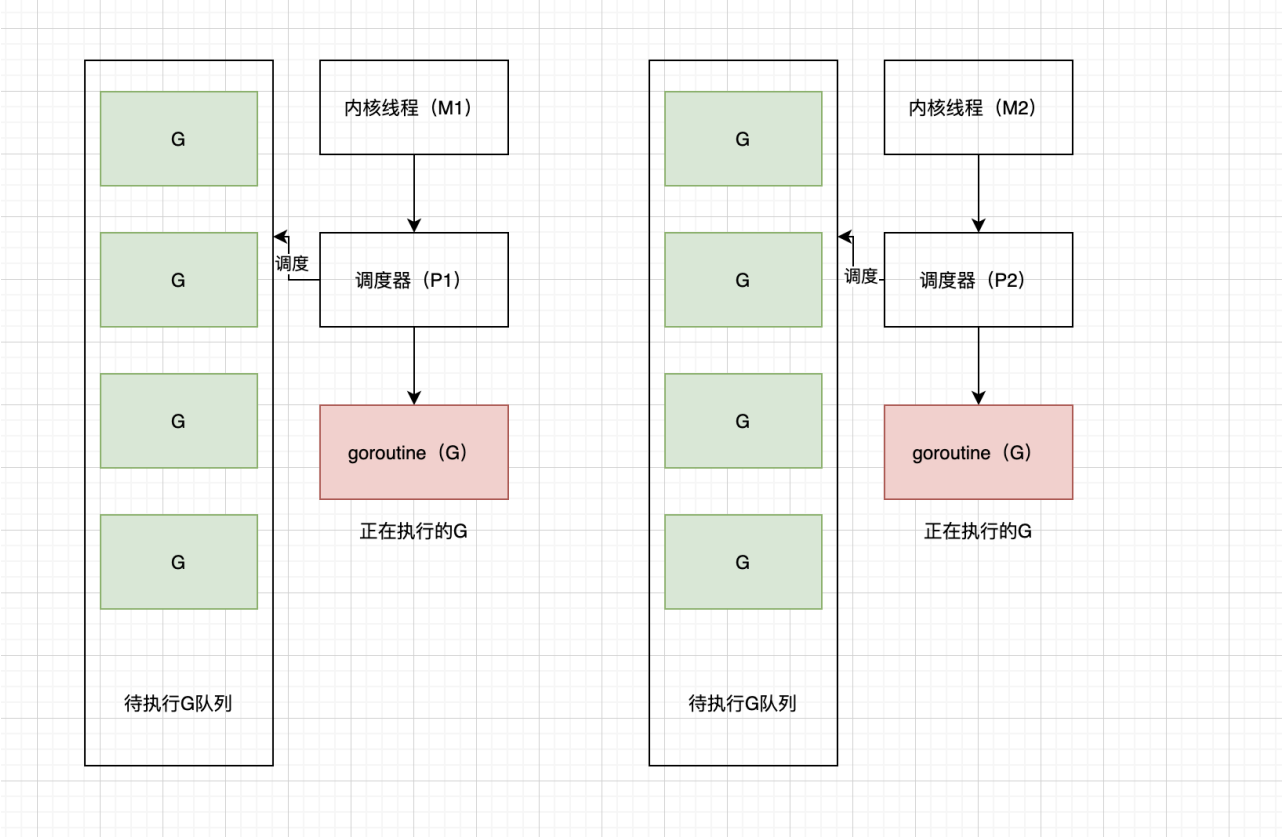
Go 的协程和 Java 的线程有啥区别?

Go 的协程和 Java 的线程确实有很大区别，我从几个方面来说一下。

首先最核心的区别是它们的调度模型不一样。Java 的线程是操作系统级别的线程，也就是我们说的内核线程，一个 Java 线程对应一个操作系统线程，它的创建、销毁、调度都是由操作系统内核来管理的。



而 Go 的协程，也就是 goroutine，它是用户态的轻量级线程，是由 Go 运行时自己调度的，不需要经过操作系统内核。这就导致了很多后续的差异。



从资源消耗上来说，差别非常明显。Java 创建一个线程的开销是比较大的，默认情况下一个线程的栈空间大概是 1MB 左右，而且线程的创建和销毁都需要系统调用，开销不小。所以在 Java 里你不能无限制地创建线程，创建几千个线程系统可能就扛不住了，这也是为什么 Java 里要用线程池来复用线程。

但 Go 的协程就轻量多了，一个 goroutine 初始栈空间只有 2KB，而且栈空间是可以动态伸缩的。Go 里创建几万甚至几十万个协程都是很正常的事情，我之前做过一个项目，同时跑了十几万个 goroutine，系统压力也不大。

调度方式上也不太一样。Java 线程的调度是抢占式的，由操作系统的调度器决定什么时候切换线程，线程切换需要保存和恢复上下文，涉及到用户态和内核态的切换，这个开销相对比较大。

Go 的协程调度是在用户态完成的，Go 运行时有个 GMP 模型，G 就是 goroutine，M 是操作系统线程，P 是逻辑处理器。Go 会把多个协程映射到少量的操作系统线程上执行，协程的切换完全在用户态，不需要陷入内核，所以切换成本非常低。而且 Go 的调度器会在协程发生阻塞的时候，比如 IO 操作或者 channel 操作时，自动把这个协程挂起，让其他协程继续执行，这种协作式调度非常高效。

从使用方式上来讲，Go 的协程用起来要简单太多了。在 Go 里创建一个协程只需要在函数调用前加个 go 关键字就行了，比如 go doSomething()，就这么简单，如下代码：

```
package main


import (
    "fmt"
    "time"
)


func doSomething(name string) {
    for i := 0; i < 3; i++ {
        fmt.Printf("%s: %d\n", name, i)
        time.Sleep(100 * time.Millisecond)
    }
}


func main() {
    // 创建协程就这么简单，加个 go 关键字
    go doSomething("协程 1")
    go doSomething("协程 2")
    go doSomething("协程 3")
```

```
    // 等待协程执行完
    time.Sleep(time.Second)
    fmt.Println("主程序结束")
}
```

而在 Java 里创建线程，你要么 new 一个 Thread 对象，要么实现 Runnable 接口，或者用线程池，总之代码量要多一些。Go 这种简洁的语法让并发编程的心智负担降低了很多。

```
public class RunnableExample {
    public static void main(String[] args) {
        // 实现 Runnable 接口
        Runnable task = new Runnable() {
            @Override
            public void run() {
                for (int i = 0; i < 3; i++) {
                    System.out.println(Thread.currentThread().getName() + ": " + i);
                    try {
                        Thread.sleep(100);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
            }
        };

        // 创建 Thread 对象并传入 Runnable
        Thread thread1 = new Thread(task, "线程 1");
        Thread thread2 = new Thread(task, "线程 2");
        Thread thread3 = new Thread(task, "线程 3");

        thread1.start();
        thread2.start();
        thread3.start();
    }
}
```

通信机制也有很大差异。Java 线程之间通信主要靠共享内存，需要用 synchronized、Lock 这些同步机制来保证线程安全，还要小心处理死锁、竞态条件这些问题，写起来比较容易出错。Go 虽然也支持共享内存和锁，但它更提倡用 channel 来通信，有句话叫"不要通过共享内存来通信，而要通过通信来共享内存"。用 channel 的话，代码会更清晰，也更不容易出现并发问题。

性能上来说，因为 Go 协程的轻量级特性和高效的调度器，在高并发场景下，Go 的表现通常会更好。比如说要处理几万个并发请求，Java 可能需要用复杂的异步框架或者线程池来优化，而 Go 直接为每个请求开一个协程就行了，代码简单性能还好。当然这不是说 Java 不行，Java 在企业级应用、生态完善性这些方面还是有很大优势的，只是在并发模型的设计上，Go 确实更现代化一些。

从我实际使用的感受来说，如果是写 IO 密集型的高并发服务，比如 API 网关、聊天服务器这种，Go 的协程模型开发效率高，性能也好。但如果是传统的企业级应用，需要用到大量的中间件和框架，Java 的生态会更成熟一些。两者各有优势，选择哪个还是要看具体场景。

并发安全

juc 包下你常用的类?

线程池相关：

- `ThreadPoolExecutor`：最核心的线程池类，用于创建和管理线程池。通过它可以灵活地配置线程池的参数，如核心线程数、最大线程数、任务队列等，以满足不同的并发处理需求。
- `Executors`：线程池工厂类，提供了一系列静态方法来创建不同类型的线程池，如 `newFixedThreadPool`（创建固定线程数的线程池）、`newCachedThreadPool`（创建可缓存线程池）、`newSingleThreadExecutor`（创建单线程线程池）等，方便开发者快速创建线程池。

并发集合类：

- `ConcurrentHashMap`：线程安全的哈希映射表，用于在多线程环境下高效地存储和访问键值对。它采用了分段锁等技术，允许多个线程同时访问不同的段，提高了并发性能，在高并发场景下比传统的 `Hashtable` 性能更好。
- `CopyOnWriteArrayList`：线程安全的列表，在对列表进行修改操作时，会创建一个新的底层数组，将修改操作应用到新数组上，而读操作仍然可以在旧数组上进行，从而实现了读写分离，提高了并发读的性能，适用于读多写少的场景。

同步工具类：

- `CountDownLatch`：允许一个或多个线程等待其他一组线程完成操作后再继续执行。它通过一个计数器来实现，计数器初始化为线程的数量，每个线程完成任务后调用 `countDown` 方法将计数

器减一，当计数器为零时，等待的线程可以继续执行。常用于多个线程完成各自任务后，再进行汇总或下一步操作的场景。

- `CyclicBarrier`：让一组线程互相等待，直到所有线程都到达某个屏障点后，再一起继续执行。与 `CountDownLatch` 不同的是，`CyclicBarrier` 可以重复使用，当所有线程都通过屏障后，计数器会重置，可以再次用于下一轮的等待。适用于多个线程需要协同工作，在某个阶段完成后再一起进入下一个阶段的场景。
- `Semaphore`：信号量，用于控制同时访问某个资源的线程数量。它维护了一个许可计数器，线程在访问资源前需要获取许可，如果有可用许可，则获取成功并将许可计数器减一，否则线程需要等待，直到有其他线程释放许可。常用于控制对有限资源的访问，如数据库连接池、线程池中的线程数量等。

原子类：

- `AtomicInteger`：原子整数类，提供了对整数类型的原子操作，如自增、自减、比较并交换等。通过硬件级别的原子指令来保证操作的原子性和线程安全性，避免了使用锁带来的性能开销，在多线程环境下对整数进行计数、状态标记等操作非常方便。
- `AtomicReference`：原子引用类，用于对对象引用进行原子操作。可以保证在多线程环境下，对对象的更新操作是原子性的，即要么全部成功，要么全部失败，不会出现数据不一致的情况。常用于实现无锁数据结构或需要对对象进行原子更新的场景。

怎么保证多线程安全?

- synchronized 关键字:可以使用 `synchronized` 关键字来同步代码块或方法，确保同一时刻只有一个线程可以访问这些代码。对象锁是通过 `synchronized` 关键字锁定对象的监视器（monitor）来实现的。

```
public synchronized void someMethod() { /* ... */ }


public void anotherMethod() {
    synchronized (someObject) {
        /* ... */
    }
}
```

- volatile 关键字:`volatile` 关键字用于变量，确保所有线程看到的是该变量的最新值，而不是可能存储在本地寄存器中的副本。

```
public volatile int sharedVariable;
```

- **Lock 接口和 ReentrantLock 类**：`java.util.concurrent.locks.Lock` 接口提供了比 `synchronized` 更强大的锁定机制，`ReentrantLock` 是一个实现该接口的例子，提供了更灵活的锁管理和更高的性能。

```
private final ReentrantLock lock = new ReentrantLock();

public void someMethod() {
    lock.lock();
    try {
        /* ... */
    } finally {
        lock.unlock();
    }
}
```

- **原子类**：Java 并发库（`java.util.concurrent.atomic`）提供了原子类，如 `AtomicInteger`、`AtomicLong` 等，这些类提供了原子操作，可以用于更新基本类型的变量而无需额外的同步。

示例：

```
AtomicInteger counter = new AtomicInteger(0);

int newValue = counter.incrementAndGet();
```

- **线程局部变量**：`ThreadLocal` 类可以为每个线程提供独立的变量副本，这样每个线程都拥有自己的变量，消除了竞争条件。

```
ThreadLocal<Integer> threadLocalVar = new ThreadLocal<>();

threadLocalVar.set(10);
int value = threadLocalVar.get();
```

- **并发集合**：使用 `java.util.concurrent` 包中的线程安全集合，如 `ConcurrentHashMap`、`ConcurrentLinkedQueue` 等，这些集合内部已经实现了线程安全的逻辑。

- **JUC 工具类**：使用 `java.util.concurrent` 包中的一些工具类可以用于控制线程间的同步和协作。例如：`Semaphore` 和 `CyclicBarrier` 等。

Java 中有哪些常用的锁，在什么场景下使用?

Java 中的锁是用于管理多线程并发访问共享资源的关键机制。锁可以确保在任意给定时间内只有一个线程可以访问特定的资源，从而避免数据竞争和不一致性。Java 提供了多种锁机制，可以分为以下几类：

- **内置锁（synchronized）**：Java 中的 `synchronized` 关键字是内置锁机制的基础，可以用于方法或代码块。当一个线程进入 `synchronized` 代码块或方法时，它会获取关联对象的锁；当线程离开该代码块或方法时，锁会被释放。如果其他线程尝试获取同一个对象的锁，它们将被阻塞，直到锁被释放。其中，syncronized 加锁时有无锁、偏向锁、轻量级锁和重量级锁几个级别。偏向锁用于当一个线程进入同步块时，如果没有任何其他线程竞争，就会使用偏向锁，以减少锁的开销。轻量级锁使用线程栈上的数据结构，避免了操作系统级别的锁。重量级锁则涉及操作系统级的互斥锁。
- **ReentrantLock**：`java.util.concurrent.locks.ReentrantLock` 是一个显式的锁类，提供了比 `synchronized` 更高级的功能，如可中断的锁等待、定时锁等待、公平锁选项等。`ReentrantLock` 使用 `lock()`和 `unlock()`方法来获取和释放锁。其中，公平锁按照线程请求锁的顺序来分配锁，保证了锁分配的公平性，但可能增加锁的等待时间。非公平锁不保证锁分配的顺序，可以减少锁的竞争，提高性能，但可能造成某些线程的饥饿。
- **读写锁（ReadWriteLock）**：`java.util.concurrent.locks.ReadWriteLock` 接口定义了一种锁，允许多个读取者同时访问共享资源，但只允许一个写入者。读写锁通常用于读取远多于写入的情况，以提高并发性。
- **乐观锁和悲观锁**：悲观锁（Pessimistic Locking）通常指在访问数据前就锁定资源，假设最坏的情况，即数据很可能被其他线程修改。`synchronized` 和 `ReentrantLock` 都是悲观锁的例子。乐观锁（Optimistic Locking）通常不锁定资源，而是在更新数据时检查数据是否已被其他线程修改。乐观锁常使用版本号或时间戳来实现。
- **自旋锁**：自旋锁是一种锁机制，线程在等待锁时会持续循环检查锁是否可用，而不是放弃 CPU 并阻塞。通常可以使用 CAS 来实现。这在锁等待时间很短的情况下可以提高性能，但过度自旋会浪费 CPU 资源。

怎么在实践中用锁的?

Java 提供了多种锁的实现，包括 `synchronized` 关键字、`java.util.concurrent.locks` 包下的 `Lock` 接口及其具体实现如 `ReentrantLock`、`ReadWriteLock` 等。下面我们来看看这些锁的使用方式。

1. `synchronized`

`synchronized` 关键字可以用于方法或代码块，它是 Java 中最早的锁实现，使用起来非常简单。

示例：synchronized 方法

```
public class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
}
```

示例：synchronized 代码块

```
public class Counter {
    private Object lock = new Object();
    private int count = 0;

    public void increment() {
        synchronized (lock) {
            count++;
        }
    }
}
```

2. 使用 `Lock` 接口

`Lock` 接口提供了比 `synchronized` 更灵活的锁操作，包括尝试锁、可中断锁、定时锁等。`ReentrantLock` 是 `Lock` 接口的一个实现。

示例：使用 ReentrantLock

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class Counter {
    private Lock lock = new ReentrantLock();
    private int count = 0;

    public void increment() {
        lock.lock();
        try {
            count++;
        } finally {
            lock.unlock();
        }
    }
}
```

3. 使用 `ReadWriteLock`

`ReadWriteLock` 接口提供了一种读写锁的实现，允许多个读操作同时进行，但写操作是独占的。

示例：使用 ReadWriteLock

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class Cache {
    private ReadWriteLock lock = new ReentrantReadWriteLock();
    private Lock readLock = lock.readLock();
    private Lock writeLock = lock.writeLock();
    private Object data;

    public Object readData() {
        readLock.lock();
```

```
        try {
            return data;
        } finally {
            readLock.unlock();
        }
    }

    public void writeData(Object newData) {
        writeLock.lock();
        try {
            data = newData;
        } finally {
            writeLock.unlock();
        }
    }
}
```

Java 并发工具你知道哪些?

Java 中一些常用的并发工具，它们位于 `java.util.concurrent` 包中，常见的有：

- CountDownLatch：CountDownLatch 是一个同步辅助类，它允许一个或多个线程等待其他线程完成操作。它使用一个计数器进行初始化，调用 `countDown()` 方法会使计数器减一，当计数器的值减为 0 时，等待的线程会被唤醒。可以把它想象成一个倒计时器，当倒计时结束（计数器为 0）时，等待的事件就会发生。示例代码：

```
import java.util.concurrent.CountDownLatch;

public class CountDownLatchExample {
    public static void main(String[] args) throws InterruptedException {
        int numberOfThreads = 3;
        CountDownLatch latch = new CountDownLatch(numberOfThreads);

        // 创建并启动三个工作线程
        for (int i = 0; i < numberOfThreads; i++) {
            new Thread(() -> {
                System.out.println(Thread.currentThread().getName() + " 正在工作");
```

```
                try {
                    Thread.sleep(1000);  // 模拟工作时间
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                latch.countDown();  // 完成工作，计数器减一
                System.out.println(Thread.currentThread().getName() + " 完成工作");
            }).start();
        }

        System.out.println("主线程等待工作线程完成");
        latch.await();  // 主线程等待，直到计数器为 0
        System.out.println("所有工作线程已完成，主线程继续执行");
    }
}
```

- **CyclicBarrier**：CyclicBarrier 允许一组线程互相等待，直到到达一个公共的屏障点。当所有线程都到达这个屏障点后，它们可以继续执行后续操作，并且这个屏障可以被重置循环使用。与 `CountDownLatch` 不同，`CyclicBarrier` 侧重于线程间的相互等待，而不是等待某些操作完成。示例代码：

```
import java.util.concurrent.CyclicBarrier;

public class CyclicBarrierExample {
    public static void main(String[] args) {
        int numberOfThreads = 3;
        CyclicBarrier barrier = new CyclicBarrier(numberOfThreads, () -> {
            System.out.println("所有线程都到达了屏障，继续执行后续操作");
        });

        for (int i = 0; i < numberOfThreads; i++) {
            new Thread(() -> {
                try {
                    System.out.println(Thread.currentThread().getName() + " 正在运行");
                    Thread.sleep(1000);  // 模拟运行时间
```

```
                    barrier.await();  // 等待其他线程
                    System.out.println(Thread.currentThread().getName() + " 已经通过屏障");
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }).start();
        }
    }
}
```

- Semaphore：Semaphore 是一个计数信号量，用于控制同时访问某个共享资源的线程数量。通过 `acquire()` 方法获取许可，使用 `release()` 方法释放许可。如果没有许可可用，线程将被阻塞，直到有许可被释放。可以用来限制对某些资源（如数据库连接池、文件操作等）的并发访问量。代码如下：

```
import java.util.concurrent.Semaphore;

public class SemaphoreExample {
    public static void main(String[] args) {
        Semaphore semaphore = new Semaphore(2);  // 允许 2 个线程同时访问

        for (int i = 0; i < 5; i++) {
            new Thread(() -> {
                try {
                    semaphore.acquire();  // 获取许可
                    System.out.println(Thread.currentThread().getName() + " 获得了许可");
                    Thread.sleep(2000);  // 模拟资源使用
                    System.out.println(Thread.currentThread().getName() + " 释放了许可");
                    semaphore.release();  // 释放许可
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }).start();
        }
    }
```

```
}
```

- **Future 和 Callable**：Callable 是一个类似于 `Runnable` 的接口，但它可以返回结果，并且可以抛出异常。Future 用于表示一个异步计算的结果，可以通过它来获取 `Callable` 任务的执行结果或取消任务。代码如下：

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class FutureCallableExample {
    public static void main(String[] args) throws Exception {
        ExecutorService executorService = Executors.newSingleThreadExecutor();

        Callable<Integer> callable = () -> {
            System.out.println(Thread.currentThread().getName() + " 开始执行 Callable 任务");
            Thread.sleep(2000);  // 模拟耗时操作
            return 42;  // 返回结果
        };

        Future<Integer> future = executorService.submit(callable);
        System.out.println("主线程继续执行其他任务");

        try {
            Integer result = future.get();  // 等待 Callable 任务完成并获取结果
            System.out.println("Callable 任务的结果: " + result);
        } catch (Exception e) {
            e.printStackTrace();
        }

        executorService.shutdown();
    }
}
```

- **ConcurrentHashMap**：ConcurrentHashMap 是一个线程安全的哈希表，它允许多个线程同时进行读操作，在一定程度上支持并发的修改操作，避免了 `HashMap` 在多线程环境下需要使用 `synchronized` 或 `Collections.synchronizedMap()` 进行同步的性能问题。代码如下：

```
import java.util.concurrent.ConcurrentHashMap;


public class ConcurrentHashMapExample {
    public static void main(String[] args) {
        ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);

        // 并发读操作
        map.forEach((key, value) -> System.out.println(key + ": " + value));

        // 并发写操作
        map.computeIfAbsent("key3", k -> 3);
    }
}
```

CountDownLatch 是做什么的讲一讲?

CountDownLatch 是 Java 并发包（`java.util.concurrent`）中的一个同步工具类，**用于让一个或多个线程等待其他线程完成操作后再继续执行**。

其核心是通过一个计数器（Counter）实现线程间的协调，常用于多线程任务的分阶段控制或主线程等待多个子线程就绪的场景，核心原理：

- **初始化计数器**：创建 `CountDownLatch` 时指定一个初始计数值（如 `N`）。
- **等待线程阻塞**：调用 `await()` 的线程会被阻塞，直到计数器变为 0。
- **任务完成通知**：其他线程完成任务后调用 `countDown()`，使计数器减 1。
- **唤醒等待线程**：当计数器减到 0 时，所有等待的线程会被唤醒。

主线程等待所有子线程就绪后启动，代码例子如下：

```
// 主线程启动多个子线程执行任务，等待全部完成后统计结果
public class MainThreadWaitExample {
```

```
    public static void main(String[] args) throws InterruptedException {
        int threadCount = 3;
        CountDownLatch latch = new CountDownLatch(threadCount);

        for (int i = 0; i < threadCount; i++) {
            new Thread(() -> {
                try {
                    System.out.println(Thread.currentThread().getName() + " 执行任务");
                    Thread.sleep(1000);
                    latch.countDown(); // 任务完成，计数器-1
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }, "Worker-" + i).start();
        }

        latch.await(); // 主线程等待所有子线程完成任务
        System.out.println("所有任务已完成");
    }
}
```

synchronized 和 reentrantlock 及其应用场景?

> synchronized 工作原理

synchronized 是 Java 提供的原子性内置锁，这种内置的并且使用者看不到的锁也被称为**监视器锁**，

使用 synchronized 之后，会在编译之后在同步的代码块前后加上 monitorenter 和 monitorexit 字节码指令，他依赖操作系统底层互斥锁实现。他的作用主要就是实现原子性操作和解决共享变量的内存可见性问题。

执行 monitorenter 指令时会尝试获取对象锁，如果对象没有被锁定或者已经获得了锁，锁的计数器+1。此时其他竞争锁的线程则会进入等待队列中。执行 monitorexit 指令时则会把计数器-1，当计数器值为 0 时，则锁释放，处于等待队列中的线程再继续竞争锁。

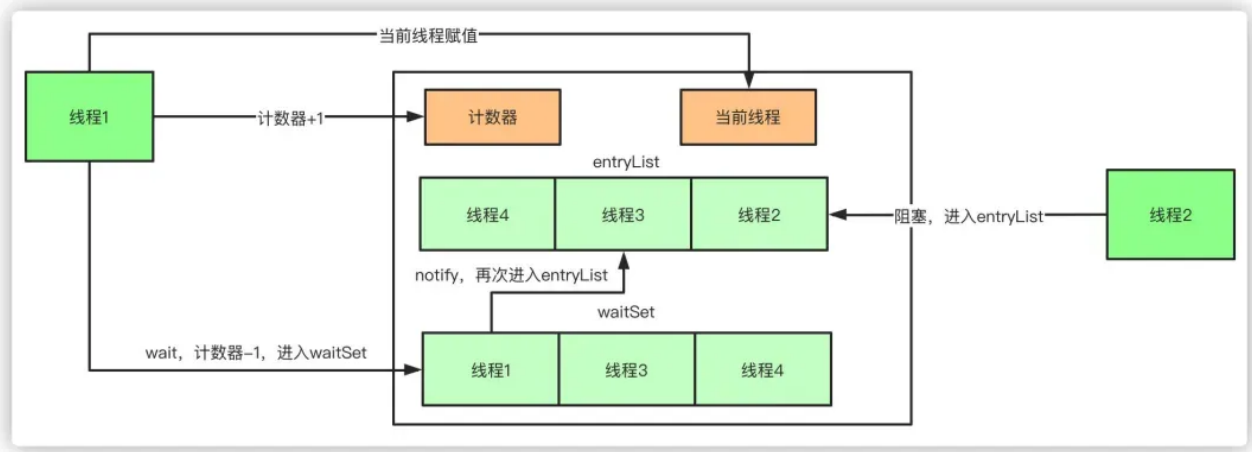
synchronized 是排它锁，当一个线程获得锁之后，其他线程必须等待该线程释放锁后才能获得锁，而且由于 Java 中的线程和操作系统原生线程是一一对应的，线程被阻塞或者唤醒时时会从用户态切换到内核态，这种转换非常消耗性能。

从内存语义来说，加锁的过程会清除工作内存中的共享变量，再从主内存读取，而释放锁的过程则是将工作内存中的共享变量写回主内存。

实际上大部分时候我认为说到 monitorenter 就行了，但是为了更清楚的描述，还是再具体一点。

如果再深入到源码来说，synchronized 实际上有两个队列 waitSet 和 entryList。

1. 当多个线程进入同步代码块时，首先进入 entryList
2. 有一个线程获取到 monitor 锁后，就赋值给当前线程，并且计数器+1
3. 如果线程调用 wait 方法，将释放锁，当前线程置为 null，计数器-1，同时进入 waitSet 等待被唤醒，调用 notify 或者 notifyAll 之后又会进入 entryList 竞争锁
4. 如果线程执行完毕，同样释放锁，计数器-1，当前线程置为 null



> reentrantlock 工作原理

ReentrantLock 的底层实现主要依赖于 AbstractQueuedSynchronizer（AQS）这个抽象类。AQS 是一个提供了基本同步机制的框架，其中包括了队列、状态值等。

ReentrantLock 在 AQS 的基础上通过内部类 Sync 来实现具体的锁操作。不同的 Sync 子类实现了公平锁和非公平锁的不同逻辑：

- **可中断性：** ReentrantLock 实现了可中断性，这意味着线程在等待锁的过程中，可以被其他线程中断而提前结束等待。在底层，ReentrantLock 使用了与 LockSupport.park() 和 LockSupport.unpark() 相关的机制来实现可中断性。

- **设置超时时间：** ReentrantLock 支持在尝试获取锁时设置超时时间，即等待一定时间后如果还未获得锁，则放弃锁的获取。这是通过内部的 tryAcquireNanos 方法来实现的。
- **公平锁和非公平锁：** 在直接创建 ReentrantLock 对象时，默认情况下是非公平锁。公平锁是按照线程等待的顺序来获取锁，而非公平锁则允许多个线程在同一时刻竞争锁，不考虑它们申请锁的顺序。公平锁可以通过在创建 ReentrantLock 时传入 true 来设置，例如：

```
ReentrantLock fairLock = new ReentrantLock(true);
```

- **多个条件变量：** ReentrantLock 支持多个条件变量，每个条件变量可以与一个 ReentrantLock 关联。这使得线程可以更灵活地进行等待和唤醒操作，而不仅仅是基于对象监视器的 wait() 和 notify()。多个条件变量的实现依赖于 Condition 接口，例如：

```
ReentrantLock lock = new ReentrantLock();
Condition condition = lock.newCondition();
// 使用下面方法进行等待和唤醒
condition.await();
condition.signal();
```

- **可重入性：** ReentrantLock 支持可重入性，即同一个线程可以多次获得同一把锁，而不会造成死锁。这是通过内部的 holdCount 计数来实现的。当一个线程多次获取锁时，holdCount 递增，释放锁时递减，只有当 holdCount 为零时，其他线程才有机会获取锁。

> 应用场景的区别

synchronized：

- **简单同步需求：** 当你需要对代码块或方法进行简单的同步控制时，`synchronized` 是一个很好的选择。它使用起来简单，不需要额外的资源管理，因为锁会在方法退出或代码块执行完毕后自动释放。
- **代码块同步：** 如果你想对特定代码段进行同步，而不是整个方法，可以使用 `synchronized` 代码块。这可以让你更精细地控制同步的范围，从而减少锁的持有时间，提高并发性能。
- **内置锁的使用：** `synchronized` 关键字使用对象的内置锁（也称为监视器锁），这在需要使用对象作为锁对象的情况下很有用，尤其是在对象状态与锁保护的代码紧密相关时。

ReentrantLock：

- **高级锁功能需求：** `ReentrantLock` 提供了 `synchronized` 所不具备的高级功能，如公平锁、响应中断、定时锁尝试、以及多个条件变量。当你需要这些功能时，`ReentrantLock` 是更好的选择。
- **性能优化：** 在高度竞争的环境中，`ReentrantLock` 可以提供比 `synchronized` 更好的性能，因为它提供了更细粒度的控制，如尝试锁定和定时锁定，可以减少线程阻塞的可能性。
- **复杂同步结构：** 当你需要更复杂的同步结构，如需要多个条件变量来协调线程之间的通信时，`ReentrantLock` 及其配套的 `Condition` 对象可以提供更灵活的解决方案。

综上，`synchronized` 适用于简单同步需求和不需要额外锁功能的场景，而 `ReentrantLock` 适用于需要更高级锁功能、性能优化或复杂同步逻辑的情况。选择哪种同步机制取决于具体的应用需求和性能考虑。

除了用 synchronized，还有什么方法可以实现线程同步?

- **使用 `ReentrantLock` 类：** `ReentrantLock` 是一个可重入的互斥锁，相比 `synchronized` 提供了更灵活的锁定和解锁操作。它还支持公平锁和非公平锁，以及可以响应中断的锁获取操作。
- **使用 `volatile` 关键字：** 虽然 `volatile` 不是一种锁机制，但它可以确保变量的可见性。当一个变量被声明为 `volatile` 后，线程将直接从主内存中读取该变量的值，这样就能保证线程间变量的可见性。但它不具备原子性。
- **使用 `Atomic` 类：** Java 提供了一系列的原子类，例如 `AtomicInteger`、`AtomicLong`、`AtomicReference` 等，用于实现对单个变量的原子操作，这些类在实现细节上利用了 CAS（Compare-And-Swap）算法，可以用来实现无锁的线程安全。

synchronized 锁静态方法和普通方法区别?

锁的对象不同：

- **普通方法：** 锁的是当前对象实例（`this`）。同一对象实例的 `synchronized` 普通方法，同一时间只能被一个线程访问；不同对象实例间互不影响，可被不同线程同时访问各自的同步普通方法。
- **静态方法：** 锁的是当前类的 `Class` 对象。由于类的 `Class` 对象全局唯一，无论多少个对象实例，该静态同步方法同一时间只能被一个线程访问。

作用范围不同：

- **普通方法：** 仅对同一对象实例的同步方法调用互斥，不同对象实例的同步普通方法可并行执行。

- **静态方法**：对整个类的所有实例的该静态方法调用都互斥，一个线程进入静态同步方法，其他线程无法进入同一类任何实例的该方法。

多实例场景影响不同：

- **普通方法**：多线程访问不同对象实例的同步普通方法时，可同时执行。
- **静态方法**：不管有多少对象实例，同一时间仅一个线程能执行该静态同步方法。

synchronized 和 reentrantlock 区别?

synchronized 和 ReentrantLock 都是 Java 中提供的可重入锁：

- **用法不同**：synchronized 可用来修饰普通方法、静态方法和代码块，而 ReentrantLock 只能用在代码块上。
- **获取锁和释放锁方式不同**：synchronized 会自动加锁和释放锁，当进入 synchronized 修饰的代码块之后会自动加锁，当离开 synchronized 的代码段之后会自动释放锁。而 ReentrantLock 需要手动加锁和释放锁
- **锁类型不同**：synchronized 属于非公平锁，而 ReentrantLock 既可以是公平锁也可以是非公平锁。
- **响应中断不同**：ReentrantLock 可以响应中断，解决死锁的问题，而 synchronized 不能响应中断。
- **底层实现不同**：synchronized 是 JVM 层面通过监视器实现的，而 ReentrantLock 是基于 AQS 实现的。

怎么理解可重入锁?

可重入锁是指同一个线程在获取了锁之后，可以再次重复获取该锁而不会造成死锁或其他问题。当一个线程持有锁时，如果再次尝试获取该锁，就会成功获取而不会被阻塞。

ReentrantLock 实现可重入锁的机制是基于线程持有锁的计数器。

- 当一个线程第一次获取锁时，计数器会加 1，表示该线程持有了锁。在此之后，如果同一个线程再次获取锁，计数器会再次加 1。每次线程成功获取锁时，都会将计数器加 1。
- 当线程释放锁时，计数器会相应地减 1。只有当计数器减到 0 时，锁才会完全释放，其他线程才有机会获取锁。

这种计数器的设计使得同一个线程可以多次获取同一个锁，而不会造成死锁或其他问题。每次获取锁时，计数器加 1；每次释放锁时，计数器减 1。只有当计数器减到 0 时，锁才会完全释放。

ReentrantLock 通过这种计数器的方式，实现了可重入锁的机制。它允许同一个线程多次获取同一个锁，并且能够正确地处理锁的获取和释放，避免了死锁和其他并发问题。

synchronized 支持重入吗？如何实现的？

synchronized 是基于原子性的内部锁机制，是可重入的，因此在一个线程调用 synchronized 方法的同时在其方法体内部调用该对象另一个 synchronized 方法，也就是说一个线程得到一个对象锁后再次请求该对象锁，是允许的，这就是 synchronized 的可重入性。

synchronized 底层是利用计算机系统 mutex Lock 实现的。每一个可重入锁都会关联一个线程 ID 和一个锁状态 status。

当一个线程请求方法时，会去检查锁状态。

1. 如果锁状态是 0，代表该锁没有被占用，使用 CAS 操作获取锁，将线程 ID 替换成自己的线程 ID。
2. 如果锁状态不是 0，代表有线程在访问该方法。此时，如果线程 ID 是自己的线程 ID，如果是可重入锁，会将 status 自增 1，然后获取到该锁，进而执行相应的方法；如果是非重入锁，就会进入阻塞队列等待。

在释放锁时，

1. 如果是可重入锁的，每一次退出方法，就会将 status 减 1，直至 status 的值为 0，最后释放该锁。
2. 如果非可重入锁的，线程退出方法，直接就会释放该锁。

syncronized 锁升级的过程讲一下

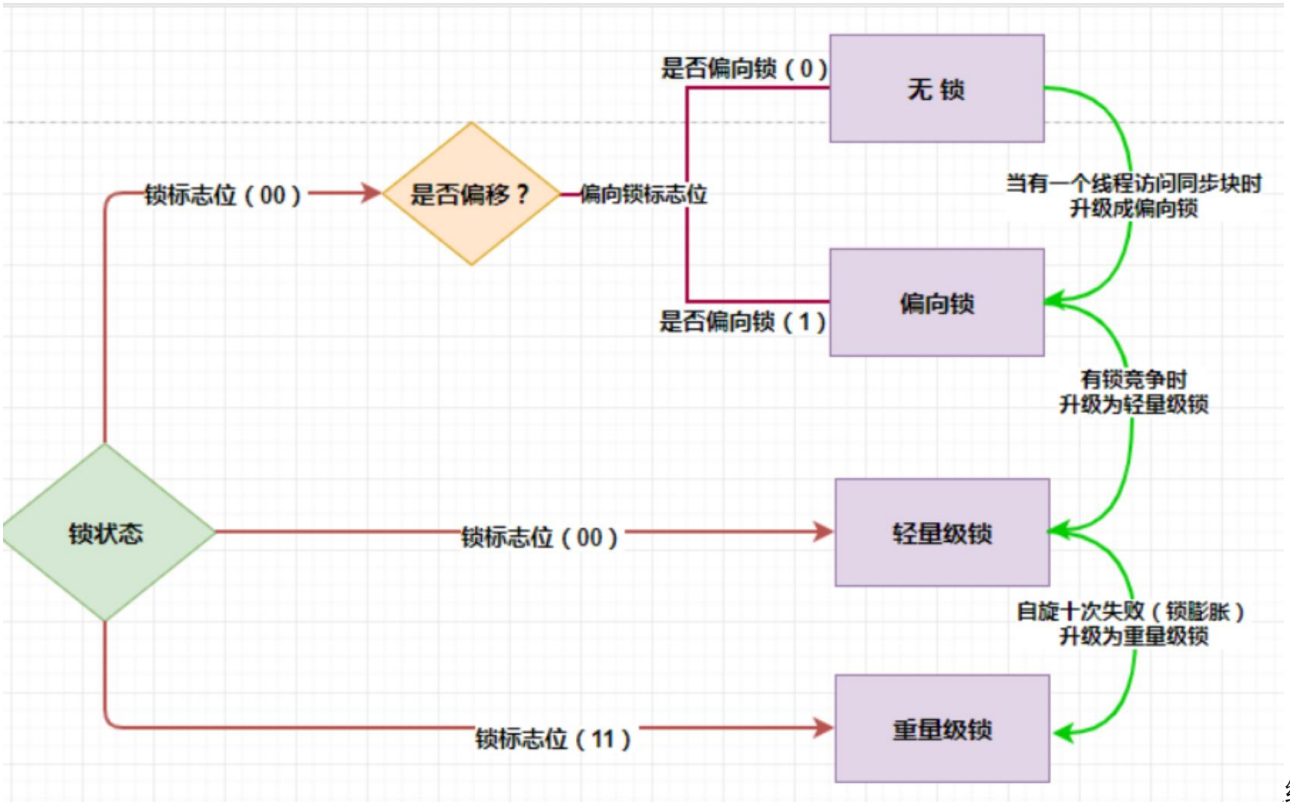
具体的锁升级的过程是：**无锁->偏向锁->轻量级锁->重量级锁**。

- **无锁**：这是没有开启偏向锁的时候的状态，在 JDK1.6 之后偏向锁的默认开启的，但是有一个偏向延迟，需要在 JVM 启动之后的多少秒之后才能开启，这个可以通过 JVM 参数进行设置，同时是否开启偏向锁也可以通过 JVM 参数设置。
- **偏向锁**：这个是在偏向锁开启之后的锁的状态，如果还没有一个线程拿到这个锁的话，这个状态叫做匿名偏向，当一个线程拿到偏向锁的时候，下次想要竞争锁只需要拿线程 ID 跟 MarkWord 当中存储的线程 ID 进行比较，如果线程 ID 相同则直接获取锁（相当于锁偏向于这个线程），不需要进行 CAS 操作和将线程挂起的操作。
- **轻量级锁**：在这个状态下线程主要是通过 CAS 操作实现的。将对象的 MarkWord 存储到线程的虚拟机栈上，然后通过 CAS 将对象的 MarkWord 的内容设置为指向 Displaced Mark Word 的指

针，如果设置成功则获取锁。在线程出临界区的时候，也需要使用 CAS，如果使用 CAS 替换成功则同步成功，如果失败表示有其他线程在获取锁，那么就需要在释放锁之后将被挂起的线程唤醒。

- **重量级锁**：当有两个以上的线程获取锁的时候轻量级锁就会升级为重量级锁，因为 CAS 如果没有成功的话始终都在自旋，进行 while 循环操作，这是非常消耗 CPU 的，但是在升级为重量级锁之后，线程会被操作系统调度然后挂起，这可以节约 CPU 资源。

了解完 4 种锁状态之后，我们就可以整体的来看一下锁升级的过程了。



线程 A 进入 synchronized 开始抢锁，JVM 会判断当前是否是偏向锁的状态，如果是就会根据 Mark Word 中存储的线程 ID 来判断，当前线程 A 是否就是持有偏向锁的线程。如果是，则忽略 check，线程 A 直接执行临界区内的代码。

但如果 Mark Word 里的线程不是线程 A，就会通过自旋尝试获取锁，如果获取到了，就将 Mark Word 中的线程 ID 改为自己的;如果竞争失败，就会立马撤销偏向锁，膨胀为轻量级锁。

后续的竞争线程都会通过自旋来尝试获取锁，如果自旋成功那么锁的状态仍然是轻量级锁。然而如果竞争失败，锁会膨胀为重量级锁，后续等待的竞争的线程都会被阻塞。

JVM 对 Synchornized 的优化?

synchronized 核心优化方案主要包含以下 4 个：

- **锁膨胀**：synchronized 从无锁升级到偏向锁，再到轻量级锁，最后到重量级锁的过程，它叫做锁膨胀也叫做锁升级。JDK 1.6 之前，synchronized 是重量级锁，也就是说 synchronized 在释放和获取锁时都会从用户态转换成内核态，而转换的效率是比较低的。但有了锁膨胀机制之后，synchronized 的状态就多了无锁、偏向锁以及轻量级锁了，这时候在进行并发操作时，大部分的场景都不需要用户态到内核态的转换了，这样就大幅的提升了 synchronized 的性能。
- **锁消除**：指的是在某些情况下，JVM 虚拟机如果检测不到某段代码被共享和竞争的可能性，就会将这段代码所属的同步锁消除掉，从而到底提高程序性能的目的。
- **锁粗化**：将多个连续的加锁、解锁操作连接在一起，扩展成一个范围更大的锁。
- **自适应自旋锁**：指通过自身循环，尝试获取锁的一种方式，优点在于它避免一些线程的挂起和恢复操作，因为挂起线程和恢复线程都需要从用户态转入内核态，这个过程是比较慢的，所以通过自旋的方式可以一定程度上避免线程挂起和恢复所造成的性能开销。

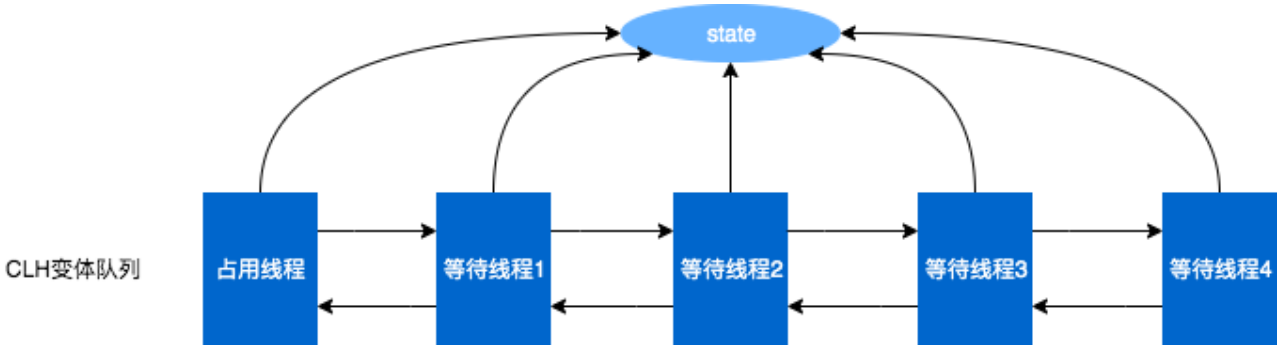
介绍一下 AQS

AQS 全称为 AbstractQueuedSynchronizer，是 Java 中的一个抽象类。 AQS 是一个用于构建锁、同步器、协作工具类的工具类（框架）。

AQS 核心思想是，如果被请求的共享资源空闲，那么就将当前请求资源的线程设置为有效的工作线程，将共享资源设置为锁定状态；如果共享资源被占用，就需要一定的阻塞等待唤醒机制来保证锁分配。这个机制主要用的是 CLH 队列的变体实现的，将暂时获取不到锁的线程加入到队列中。

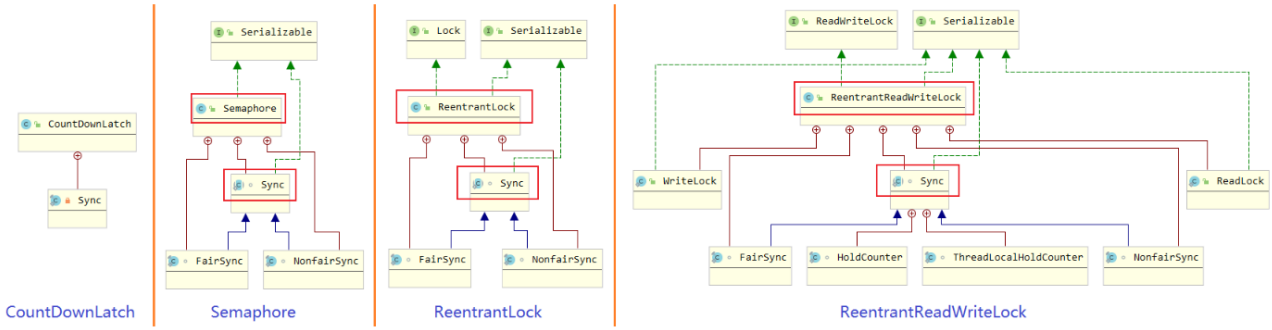
CLH：Craig、Landin and Hagersten 队列，是单向链表，AQS 中的队列是 CLH 变体的虚拟双向队列（FIFO），AQS 是通过将每条请求共享资源的线程封装成一个节点来实现锁的分配。

主要原理图如下：

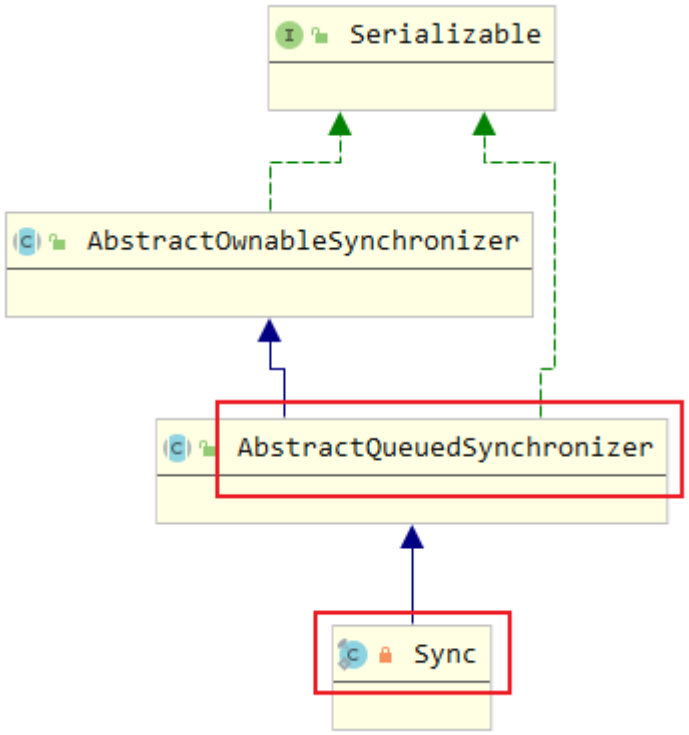


AQS 使用一个 Volatile 的 int 类型的成员变量来表示同步状态，通过内置的 FIFO 队列来完成资源获取的排队工作，通过 CAS 完成对 State 值的修改。

AQS 广泛用于控制并发流程的类，如下图：



其中 `Sync` 是这些类中都有的内部类，其结构如下：



可以看到：`Sync` 是 `AQS` 的实现。 `AQS` 主要完成的任务：

- 同步状态（比如说计数器）的原子性管理；
- 线程的阻塞和解除阻塞；
- 队列的管理。

> AQS 原理

AQS 最核心的就是三大部分：

- 状态：state；
- 控制线程抢锁和配合的 FIFO 队列（双向链表）；
- 期望协作工具类去实现的获取/释放等重要方法（重写）。

状态 state

- 这里 state 的具体含义，会根据具体实现类的不同而不同：比如在 Semapore 里，他表示剩余许可证的数量；在 CountDownLatch 里，它表示还需要倒数的数量；在 ReentrantLock 中，state 用来表示“锁”的占有情况，包括可重入计数，当 state 的值为 0 的时候，标识该 Lock 不被任何线程所占有。
- state 是 volatile 修饰的，并被并发修改，所以修改 state 的方法都需要保证线程安全，比如 getState、setState 以及 compareAndSetState 操作来读取和更新这个状态。这些方法都依赖于 unsafe 类。

FIFO 队列

- 这个队列用来存放“等待的线程，AQS 就是“排队管理器”，当多个线程争用同一把锁时，必须有排队机制将那些没能拿到锁的线程串在一起。当锁释放时，锁管理器就会挑选一个合适的线程来占有这个刚刚释放的锁。
- AQS 会维护一个等待的线程队列，把线程都放到这个队列里，这个队列是双向链表形式。

实现获取/释放等方法

- 这里的获取和释放方法，是利用 AQS 的协作工具类里最重要的方法，是由协作类自己去实现的，并且含义各不相同；
- 获取方法：获取操作会以来 state 变量，经常会阻塞（比如获取不到锁的时候）。在 Semaphore 中，获取就是 acquire 方法，作用是获取一个许可证； 而在 CountDownLatch 里面，获取就是 await 方法，作用是等待，直到倒数结束；
- 释放方法：在 Semaphore 中，释放就是 release 方法，作用是释放一个许可证； 在 CountDownLatch 里面，获取就是 countDown 方法，作用是将倒数的数减一；
- 需要每个实现类重写 tryAcquire 和 tryRelease 等方法。

CAS 和 AQS 有什么关系?

CAS 和 AQS 两者的区别：

- CAS 是一种乐观锁机制，它包含三个操作数：内存位置（V）、预期值（A）和新值（B）。CAS 操作的逻辑是，如果内存位置 V 的值等于预期值 A，则将其更新为新值 B，否则不做任何操作。整个过程是原子性的，通常由硬件指令支持，如在现代处理器上，`cmpxchg` 指令可以实现 CAS 操作。
- AQS 是一个用于构建锁和同步器的框架，许多同步器如 `ReentrantLock`、`Semaphore`、`CountDownLatch` 等都是基于 AQS 构建的。AQS 使用一个 `volatile` 的整数变量 `state` 来表示同步状态，通过内置的 `FIFO` 队列来管理等待线程。它提供了一些基本的操作，如 `acquire`

（获取资源）和 `release`（释放资源），这些操作会修改 `state` 的值，并根据 `state` 的值来判断线程是否可以获取或释放资源。AQS 的 `acquire` 操作通常会先尝试获取资源，如果失败，线程将被添加到等待队列中，并阻塞等待。`release` 操作会释放资源，并唤醒等待队列中的线程。

CAS 和 AQS 两者的联系：

- **CAS 为 AQS 提供原子操作支持**：AQS 内部使用 CAS 操作来更新 `state` 变量，以实现线程安全的状态修改。在 `acquire` 操作中，当线程尝试获取资源时，会使用 CAS 操作尝试将 `state` 从一个值更新为另一个值，如果更新失败，说明资源已被占用，线程会进入等待队列。在 `release` 操作中，当线程释放资源时，也会使用 CAS 操作将 `state` 恢复到相应的值，以保证状态更新的原子性。

如何用 AQS 实现一个可重入的公平锁?

AQS 实现一个可重入的公平锁的详细步骤：

1. **继承 AbstractQueuedSynchronizer**：创建一个内部类继承自 `AbstractQueuedSynchronizer`，重写 `tryAcquire`、`tryRelease`、`isHeldExclusively` 等方法，这些方法将用于实现锁的获取、释放和判断锁是否被当前线程持有。
2. **实现可重入逻辑**：在 `tryAcquire` 方法中，检查当前线程是否已经持有锁，如果是，则增加锁的持有次数（通过 `state` 变量）；如果不是，尝试使用 CAS 操作来获取锁。
3. **实现公平性**：在 `tryAcquire` 方法中，按照队列顺序来获取锁，即先检查等待队列中是否有线程在等待，如果有，当前线程必须进入队列等待，而不是直接竞争锁。
4. **创建锁的外部类**：创建一个外部类，内部持有 `AbstractQueuedSynchronizer` 的子类对象，并提供 `lock` 和 `unlock` 方法，这些方法将调用 `AbstractQueuedSynchronizer` 子类中的方法。

```
import java.util.concurrent.locks.AbstractQueuedSynchronizer;


public class FairReentrantLock {


    private static class Sync extends AbstractQueuedSynchronizer {


        // 判断锁是否被当前线程持有
        protected boolean isHeldExclusively() {
            return getExclusiveOwnerThread() == Thread.currentThread();
        }
```

```
// 尝试获取锁
protected boolean tryAcquire(int acquires) {
    final Thread current = Thread.currentThread();
    int c = getState();
    if (c == 0) {
        // 公平性检查：检查队列中是否有前驱节点，如果有，则当前线程不能获取锁
        if (!hasQueuedPredecessors() && compareAndSetState(0, acquires)) {
            setExclusiveOwnerThread(current);
            return true;
        }
    } else if (current == getExclusiveOwnerThread()) {
        // 可重入逻辑：如果是当前线程持有锁，则增加持有次数
        int nextc = c + acquires;
        if (nextc < 0) {
            throw new Error("Maximum lock count exceeded");
        }
        setState(nextc);
        return true;
    }
    return false;
}

// 尝试释放锁
protected boolean tryRelease(int releases) {
    int c = getState() - releases;
    if (Thread.currentThread()!= getExclusiveOwnerThread()) {
        throw new IllegalMonitorStateException();
    }
    boolean free = false;
    if (c == 0) {
        free = true;
        setExclusiveOwnerThread(null);
    }
    setState(c);
```

```
            return free;
        }

        // 提供一个条件变量，用于实现更复杂的同步需求，这里只是简单实现
        ConditionObject newCondition() {
            return new ConditionObject();
        }
    }

    private final Sync sync = new Sync();

    // 加锁方法
    public void lock() {
        sync.acquire(1);
    }

    // 解锁方法
    public void unlock() {
        sync.release(1);
    }

    // 判断当前线程是否持有锁
    public boolean isLocked() {
        return sync.isHeldExclusively();
    }

    // 提供一个条件变量，用于实现更复杂的同步需求，这里只是简单实现
    public Condition newCondition() {
        return sync.newCondition();
    }
}
```

代码解释：

内部类 Sync：

- `isHeldExclusively`：使用 `getExclusiveOwnerThread` 方法检查当前锁是否被当前线程持有。

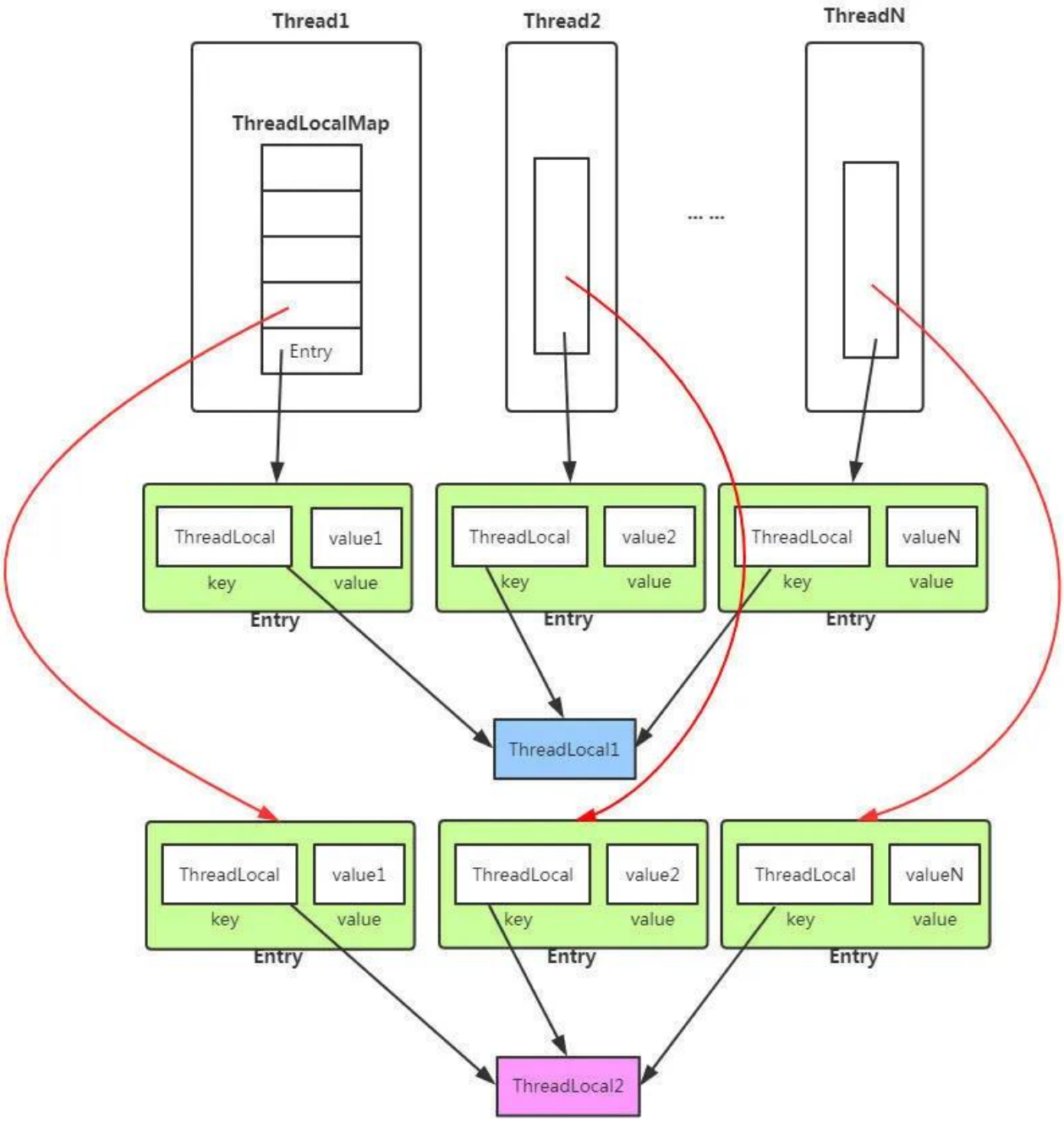
- `tryAcquire`：
    - 首先获取当前锁的状态 `c`。
    - 如果 `c` 为 0，表示锁未被持有，此时进行公平性检查，通过 `hasQueuedPredecessors` 检查是否有前驱节点在等待队列中。如果没有，使用 `compareAndSetState` 尝试将状态设置为 `acquires`（通常为 1），并设置当前线程为锁的持有线程。
    - 如果 `c` 不为 0，说明锁已被持有，检查是否为当前线程持有。如果是，增加锁的持有次数（可重入），但要防止溢出。
- `tryRelease`：
    - 先将状态减 `releases`（通常为 1）。
    - 检查当前线程是否为锁的持有线程，如果不是，抛出异常。
    - 如果状态减为 0，说明锁被完全释放，将持有线程设为 `null`。
- `newCondition`：创建一个 `ConditionObject` 用于更复杂的同步操作，如等待 / 通知机制。

外部类 FairReentrantLock：

- `lock` 方法：调用 `sync.acquire(1)` 尝试获取锁。
- `unlock` 方法：调用 `sync.release(1)` 释放锁。
- `isLocked` 方法：调用 `sync.isHeldExclusively` 判断锁是否被当前线程持有。
- `newCondition` 方法：调用 `sync.newCondition` 提供条件变量。

Threadlocal 作用，原理，具体里面存的 key value 是啥，会有什么问题，如何解决?

`ThreadLocal` 是 Java 中用于解决线程安全问题的一种机制，它允许创建线程局部变量，即每个线程都有自己独立的变量副本，从而避免了线程间的资源共享和同步问题。



从内存结构图，我们可以看到：

- Thread 类中，有个 ThreadLocal.ThreadLocalMap 的成员变量。
- ThreadLocalMap 内部维护了 Entry 数组，每个 Entry 代表一个完整的对象，key 是 ThreadLocal 本身，value 是 ThreadLocal 的泛型对象值。

> ThreadLocal 的作用

- **线程隔离**：`ThreadLocal` 为每个线程提供了独立的变量副本，这意味着线程之间不会相互影响，可以安全地在多线程环境中使用这些变量而不必担心数据竞争或同步问题。
- **降低耦合度**：在同一个线程内的多个函数或组件之间，使用 `ThreadLocal` 可以减少参数的传递，降低代码之间的耦合度，使代码更加清晰和模块化。

- **性能优势**：由于 `ThreadLocal` 避免了线程间的同步开销，所以在大量线程并发执行时，相比传统的锁机制，它可以提供更好的性能。

> ThreadLocal 的原理

`ThreadLocal` 的实现依赖于 `Thread` 类中的一个 `ThreadLocalMap` 字段，这是一个存储 `ThreadLocal` 变量本身和对应值的映射。每个线程都有自己的 `ThreadLocalMap` 实例，用于存储该线程所持有的所有 `ThreadLocal` 变量的值。

当你创建一个 `ThreadLocal` 变量时，它实际上就是一个 `ThreadLocal` 对象的实例。每个 `ThreadLocal` 对象都可以存储任意类型的值，这个值对每个线程来说是独立的。

- 当调用 `ThreadLocal` 的 `get()`方法时，`ThreadLocal` 会检查当前线程的 `ThreadLocalMap` 中是否有与之关联的值。
- 如果有，返回该值；
- 如果没有，会调用 `initialValue()`方法（如果重写了的话）来初始化该值，然后将其放入 `ThreadLocalMap` 中并返回。
- 当调用 `set()`方法时，`ThreadLocal` 会将给定的值与当前线程关联起来，即在当前线程的 `ThreadLocalMap` 中存储一个键值对，键是 `ThreadLocal` 对象自身，值是传入的值。
- 当调用 `remove()`方法时，会从当前线程的 `ThreadLocalMap` 中移除与该 `ThreadLocal` 对象关联的条目。

> 可能存在的问题

当一个线程结束时，其 `ThreadLocalMap` 也会随之销毁，但是 `ThreadLocal` 对象本身不会立即被垃圾回收，直到没有其他引用指向它为止。

因此，在使用 `ThreadLocal` 时需要注意，**如果不显式调用 `remove()`方法，或者线程结束时未正确清理 `ThreadLocal` 变量，可能会导致内存泄漏，因为 `ThreadLocalMap` 会持续持有 `ThreadLocal` 变量的引用，即使这些变量不再被其他地方引用。**

因此，实际应用中需要在使用完 `ThreadLocal` 变量后调用 `remove()`方法释放资源。

悲观锁和乐观锁的区别?

- 乐观锁： 就像它的名字一样，对于并发间操作产生的线程安全问题持乐观状态，乐观锁认为竞争不总 是会发生，因此它不需要持有锁，将比较-替换这两个动作作为一个原子操作尝试去修改内存中的变量，如果失败则表示发生冲突，那么就应该有相应的重试逻辑。

- 悲观锁： 还是像它的名字一样，对于并发间操作产生的线程安全问题持悲观状态，悲观锁认为竞争总 是会发生，因此每次对某资源进行操作时，都会持有一个独占的锁，就像 synchronized，不管三七二十一，直接上了锁就操作资源了。

Java 中想实现一个乐观锁，都有哪些方式?

1. **CAS（Compare and Swap）操作：** CAS 是乐观锁的基础。Java 提供了 java.util.concurrent.atomic 包，包含各种原子变量类（如 AtomicInteger、AtomicLong），这些类使用 CAS 操作实现了线程安全的原子操作，可以用来实现乐观锁。
2. **版本号控制**：增加一个版本号字段记录数据更新时候的版本，每次更新时递增版本号。在更新数据时，同时比较版本号，若当前版本号和更新前获取的版本号一致，则更新成功，否则失败。
3. **时间戳**：使用时间戳记录数据的更新时间，在更新数据时，在比较时间戳。如果当前时间戳大于数据的时间戳，则说明数据已经被其他线程更新，更新失败。

CAS 有什么缺点?

CAS 的缺点主要有 3 点：

- **ABA 问题**：ABA 的问题指的是在 CAS 更新的过程中，当读取到的值是 A，然后准备赋值的时候仍然是 A，但是实际上有可能 A 的值被改成了 B，然后又被改回了 A，这个 CAS 更新的漏洞就叫做 ABA。只是 ABA 的问题大部分场景下都不影响并发的最终效果。Java 中有 AtomicStampedReference 来解决这个问题，他加入了预期标志和更新后标志两个字段，更新时不光检查值，还要检查当前的标志是否等于预期标志，全部相等的话才会更新。
- **循环时间长开销大**：自旋 CAS 的方式如果长时间不成功，会给 CPU 带来很大的开销。
- **只能保证一个共享变量的原子操作**：只对一个共享变量操作可以保证原子性，但是多个则不行，多个可以通过 AtomicReference 来处理或者使用锁 synchronized 实现。

为什么不能所有的锁都用 CAS?

CAS 操作是基于循环重试的机制，如果 CAS 操作一直未能成功，线程会一直自旋重试，占用 CPU 资源。在高并发情况下，大量线程自旋会导致 CPU 资源浪费。

CAS 有什么问题，Java 是怎么解决的?

会有 ABA 的问题，变量值在操作过程中先被其他线程从 **A** 修改为 **B**，又被改回 **A**，CAS 无法感知中途变化，导致操作误判为“未变更”。比如：

- 线程 1 读取变量为 `A`，准备改为 `C`。
- 此时线程 2 将变量 `A→B→A`。
- 线程 1 的 CAS 执行时发现仍是 `A`，但状态已丢失中间变化。

Java 提供的工具类会在 CAS 操作中增加**版本号（Stamp）或标记**，每次修改都更新版本号，使得即使值相同也能识别变更历史。比如，可以用 AtomicStampedReference 来解决 ABA 问题，通过比对值**和**版本号识别 ABA 问题。

```
AtomicStampedReference<Integer> ref = new AtomicStampedReference<>(100, 0);


// 尝试修改值并更新版本号
boolean success = ref.compareAndSet(100, 200, 0, 1);
// 前提：当前值=100 且 版本号=0，才会更新为（200,1）
```

voliatle 关键字有什么作用?

volatite 作用有 2 个：

- **保证变量对所有线程的可见性**。当一个变量被声明为 volatile 时，它会保证对这个变量的写操作会立即刷新到主存中，而对这个变量的读操作会直接从主存中读取，从而确保了多线程环境下对该变量访问的可见性。这意味着一个线程修改了 volatile 变量的值，其他线程能够立刻看到这个修改，不会受到各自线程工作内存的影响。
- **禁止指令重排序优化**。volatile 关键字在 Java 中主要通过内存屏障来禁止特定类型的指令重排序。
    - 1）**写-写（Write-Write）屏障**：在对 volatile 变量执行写操作之前，会插入一个写屏障。这确保了在该变量写操作之前的所有普通写操作都已完成，防止了这些写操作被移到 volatile 写操作之后。
    - 2）**读-写（Read-Write）屏障**：在对 volatile 变量执行读操作之后，会插入一个读屏障。它确保了对 volatile 变量的读操作之后的所有普通读操作都不会被提前到 volatile 读之前执行，保证了读取到的数据是最新的。
    - 3）**写-读（Write-Read）屏障**：这是最重要的一个屏障，它发生在 volatile 写之后和 volatile 读之前。这个屏障确保了 volatile 写操作之前的所有内存操作（包括写操作）都不会被重排序到 volatile 读之后，同时也确保了 volatile 读操作之后的所有内存操作（包括读操作）都不会被重排序到 volatile 写之前。

指令重排序的原理是什么?

在执行程序时，为了提高性能，处理器和编译器常常会对指令进行重排序，但是重排序要满足下面 2 个条件才能进行：

- 在单线程环境下不能改变程序运行的结果
- 存在数据依赖关系的不允许重排序。

所以重排序不会对单线程有影响，只会破坏多线程的执行语义。

我们看这个例子，A 和 C 之间存在数据依赖关系，同时 B 和 C 之间也存在数据依赖关系。因此在最终执行的指令序列中，C 不能被重排序到 A 和 B 的前面，如果 C 排到 A 和 B 的前面，那么程序的结果将会被改变。但 A 和 B 之间没有数据依赖关系，编译器和处理器可以重排序 A 和 B 之间的执行顺序。

```
1  double width  = 15.67;          //A
2  double height = 14.32;          //B
3  double area = width * height;   //C
```

volatile 可以保证线程安全吗?

volatile 关键字可以保证可见性，但不能保证原子性，因此不能完全保证线程安全。volatile 关键字用于修饰变量，当一个线程修改了 volatile 修饰的变量的值，其他线程能够立即看到最新的值，从而避免了线程之间的数据不一致。

但是，volatile 并不能解决多线程并发下的复合操作问题，比如 i++这种操作不是原子操作，如果多个线程同时对 i 进行自增操作，volatile 不能保证线程安全。对于复合操作，需要使用 synchronized 关键字或者 Lock 来保证原子性和线程安全。

volatile 和 sychronized 比较?

Synchronized 解决了多线程访问共享资源时可能出现的竞态条件和数据不一致的问题，保证了线程安全性。Volatile 解决了变量在多线程环境下的可见性和有序性问题，确保了变量的修改对其他线程是可见的。

- Synchronized: Synchronized 是一种排他性的同步机制，保证了多个线程访问共享资源时的互斥性，即同一时刻只允许一个线程访问共享资源。通过对代码块或方法添加 Synchronized 关键字来实现同步。

- Volatile: Volatile 是一种轻量级的同步机制，用来保证变量的可见性和禁止指令重排序。当一个变量被声明为 Volatile 时，线程在读取该变量时会直接从内存中读取，而不会使用缓存，同时对该变量的写操作会立即刷回主内存，而不是缓存在本地内存中。

什么是公平锁和非公平锁?

- **公平锁：** 指多个线程按照申请锁的顺序来获取锁，线程直接进入队列中排队，队列中的第一个线程才能获得锁。公平锁的优点在于各个线程公平平等，每个线程等待一段时间后，都有执行的机会，而它的缺点就在于整体执行速度更慢，吞吐量更小。
- **非公平锁：** 多个线程加锁时直接尝试获取锁，能抢到锁到直接占有锁，抢不到才会到等待队列的队尾等待。非公平锁的优势就在于整体执行速度更快，吞吐量更大，但同时也可能产生线程饥饿问题，也就是说如果一直有线程插队，那么在等待队列中的线程可能长时间得不到运行。

非公平锁吞吐量为什么比公平锁大?

- **公平锁执行流程**：获取锁时，先将线程自己添加到等待队列的队尾并休眠，当某线程用完锁之后，会去唤醒等待队列中队首的线程尝试去获取锁，锁的使用顺序也就是队列中的先后顺序，在整个过程中，线程会从运行状态切换到休眠状态，再从休眠状态恢复成运行状态，但线程每次休眠和恢复都需要从用户态转换成内核态，而这个状态的转换是比较慢的，所以公平锁的执行速度会比较慢。
- **非公平锁执行流程**：当线程获取锁时，会先通过 CAS 尝试获取锁，如果获取成功就直接拥有锁，如果获取锁失败才会进入等待队列，等待下次尝试获取锁。这样做的好处是，获取锁不用遵循先到先得的规则，从而避免了线程休眠和恢复的操作，这样就加速了程序的执行效率。

Synchronized 是公平锁吗?

Synchronized 不属于公平锁，ReentrantLock 是公平锁。

ReentrantLock 是怎么实现公平锁的?

我们来看一下公平锁与非公平锁的加锁方法的源码。公平锁的锁获取源码如下：

```
protected final boolean tryAcquire(int acquires) {


    final Thread current = Thread.currentThread();
    int c = getState();
```

```
    if (c == 0) {

        if (!hasQueuedPredecessors() && //这里判断了 hasQueuedPredecessors()
                compareAndSetState(0, acquires)) {

            setExclusiveOwnerThread(current);

            return true;
        }

    } else if (current == getExclusiveOwnerThread()) {

        int nextc = c + acquires;

        if (nextc < 0) {
            throw new Error("Maximum lock count exceeded");
        }
        setState(nextc);
        return true;

    }
    return false;
}
```

非公平锁的锁获取源码如下：

```
final boolean nonfairTryAcquire(int acquires) {

    final Thread current = Thread.currentThread();
    int c = getState();

    if (c == 0) {

        if (compareAndSetState(0, acquires)) { //这里没有判断      hasQueuedPredecessors()
```

```
            setExclusiveOwnerThread(current);

            return true;

        }

    }

    else if (current == getExclusiveOwnerThread()) {

        int nextc = c + acquires;

        if (nextc < 0) // overflow

        throw new Error("Maximum lock count exceeded");

        setState(nextc);

        return true;

    }

    return false;

}
```

通过对比，我们可以明显的看出公平锁与非公平锁的 lock() 方法唯一的区别就在于公平锁在获取锁时多了一个限制条件：hasQueuedPredecessors() 为 false，这个方法就是判断在等待队列中是否已经有线程在排队了。

这也就是公平锁和非公平锁的核心区别，如果是公平锁，那么一旦已经有线程在排队了，当前线程就不再尝试获取锁；对于非公平锁而言，无论是否已经有线程在排队，都会尝试获取一下锁，获取不到的话，再去排队。这里有一个特例需要我们注意，针对 tryLock() 方法，它不遵守设定的公平原则。

例如，当有线程执行 tryLock() 方法的时候，一旦有线程释放了锁，那么这个正在 tryLock 的线程就能获取到锁，即使设置的是公平锁模式，即使在它之前已经有其他正在等待队列中等待的线程，简单地说就是 tryLock 可以插队。

看它的源码就会发现：

```
public boolean tryLock() {

    return sync.nonfairTryAcquire(1);

}
```

这里调用的就是 nonfairTryAcquire()，表明了是不公平的，和锁本身是否是公平锁无关。综上所述，公平锁就是会按照多个线程申请锁的顺序来获取锁，从而实现公平的特性。

非公平锁加锁时不考虑排队等待情况，直接尝试获取锁，所以存在后申请却先获得锁的情况，但由此也提高了整体的效率。

什么情况会产生死锁问题？如何解决？

死锁只有**同时满足**以下四个条件才会发生：

- 互斥条件：互斥条件是指**多个线程不能同时使用同一个资源**。
- 持有并等待条件：持有并等待条件是指，当线程 A 已经持有了资源 1，又想申请资源 2，而资源 2 已经被线程 C 持有了，所以线程 A 就会处于等待状态，但是**线程 A 在等待资源 2 的同时并不会释放自己已经持有的资源 1**。
- 不可剥夺条件：不可剥夺条件是指，当线程已经持有了资源 ，**在自己使用完之前不能被其他线程获取**，线程 B 如果也想使用此资源，则只能在线程 A 使用完并释放后才能获取。
- 环路等待条件：环路等待条件指的是，在死锁发生的时候，**两个线程获取资源的顺序构成了环形链**。

例如，线程 A 持有资源 R1 并试图获取资源 R2，而线程 B 持有资源 R2 并试图获取资源 R1，此时两个线程相互等待对方释放资源，从而导致死锁。

```
public class DeadlockExample {
    private static final Object resource1 = new Object();
    private static final Object resource2 = new Object();
```

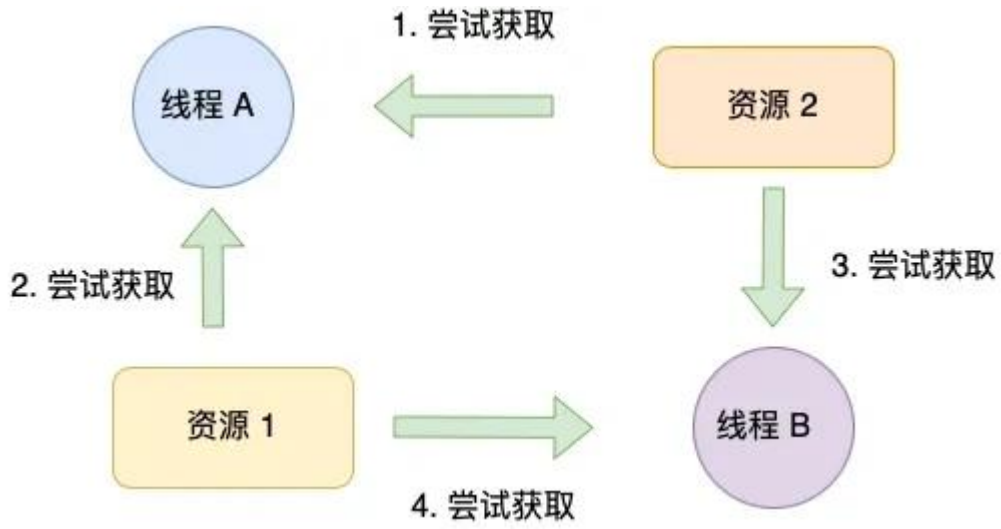
```
    public static void main(String[] args) {
        Thread threadA = new Thread(() -> {
            synchronized (resource1) {
                System.out.println("Thread A acquired resource1");
                try {
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                synchronized (resource2) {
                    System.out.println("Thread A acquired resource2");
                }
            }
        });

        Thread threadB = new Thread(() -> {
            synchronized (resource2) {
                System.out.println("Thread B acquired resource2");
                try {
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                synchronized (resource1) {
                    System.out.println("Thread B acquired resource1");
                }
            }
        });

        threadA.start();
        threadB.start();
    }
}
```

避免死锁问题就只需要破坏其中一个条件就可以，最常见的并且可行的就是**使用资源有序分配法，来破坏环路等待条件**。

那什么是资源有序分配法呢？线程 A 和 线程 B 获取资源的顺序要一样，当线程 A 是先尝试获取资源 A，然后尝试获取资源 B 的时候，线程 B 同样也是先尝试获取资源 A，然后尝试获取资源 B。也就是说，线程 A 和 线程 B 总是以相同的顺序申请自己想要的资源。



线程池

线程池怎么使用?

线程池的核心作用是复用线程、控制并发数，避免频繁创建销毁线程带来的性能开销。

首先最常用的是通过 Executors 工具类快速创建线程池，适合简单场景，不用手动配置复杂参数。比如需要一个固定线程数的线程池，处理一批异步任务：

```
// 创建固定 3 个线程的线程池
ExecutorService fixedThreadPool = Executors.newFixedThreadPool(3);

// 提交 Runnable 任务 无返回值
for (int i = 0; i < 5; i++) {
    int taskNum = i;
    fixedThreadPool.submit(() -> {
        System.out.println(当前线程  + Thread.currentThread().getName() +  处理任务 + taskNum);
        try {
            Thread.sleep(1000); // 模拟任务执行耗时
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });
```

```
}

// 提交 Callable 任务 有返回值
Future<Integer> future = fixedThreadPool.submit(() -> {
    Thread.sleep(500);
    return100 + 200; // 任务返回结果
});

// 获取 Callable 任务的返回值 会阻塞直到任务完成
try {
    Integer result = future.get();
    System.out.println(任务返回结果 + result);
} catch (Exception e) {
    e.printStackTrace();
}

// 任务执行完后关闭线程池 避免资源泄露
fixedThreadPool.shutdown();
```

不过 Executors 创建的线程池有潜在问题，比如 newCachedThreadPool 可能创建大量线程导致 OOM，newScheduledThreadPool 的核心线程数默认无界，所以复杂业务场景更推荐用 ThreadPoolExecutor 手动配置，能精准控制线程池行为。

手动创建 ThreadPoolExecutor 需要指定 7 个核心参数，理解这些参数才能用好。比如创建一个适合处理 IO 密集型任务的线程池（IO 密集型任务线程数可以多些，一般是 CPU 核心数的 2 倍）：

```
// 获取 CPU 核心数 用于合理设置线程数
int corePoolSize = Runtime.getRuntime().availableProcessors() * 2;

// 手动配置线程池
ThreadPoolExecutor threadPool = new ThreadPoolExecutor(
    corePoolSize, // 核心线程数 线程池长期维持的最小线程数
    corePoolSize * 2, // 最大线程数 线程池能容纳的最多线程数
    60L, // 空闲线程存活时间 超过核心线程数的空闲线程 多久后销毁
    TimeUnit.SECONDS, // 存活时间单位
    new ArrayBlockingQueue<>(100), // 任务阻塞队列 核心线程忙时 新任务存这里
```

```
    Executors.defaultThreadFactory(), // 线程创建工厂 用于设置线程名 优先级等
    new ThreadPoolExecutor.AbortPolicy() // 拒绝策略 队列满且线程数达最大时 如何处理新任务
);

// 提交任务的两种方式和之前一致 这里用 execute 提交 Runnable（无返回值 不能捕获异常）
threadPool.execute(() -> {
    System.out.println(IO 任务执行中 + Thread.currentThread().getName());
    // 模拟 IO 操作 比如数据库查询 网络请求
});

// 关闭线程池 推荐用 shutdown 等待已提交任务完成后再关闭
threadPool.shutdown();
// 若需要强制关闭 可调用 shutdownNow 会中断正在执行的任务 返回未执行的任务
// List<Runnable> unExecutedTasks = threadPool.shutdownNow();
```

这里要注意几个关键细节：比如拒绝策略有四种，除了默认的 AbortPolicy（直接抛异常），还有 CallerRunsPolicy（让提交任务的主线程自己执行，缓解压力）、DiscardPolicy（直接丢弃新任务）、DiscardOldestPolicy（丢弃队列里最旧的任务，再提交新任务），要根据业务选择，比如核心业务不能丢任务，就别用 Discard 相关策略。

另外，提交任务时，submit 和 execute 的区别在于 submit 能提交 Callable 有返回值，还能通过 Future 捕获任务执行中的异常，而 execute 只能提交 Runnable，异常会直接抛出，比如：

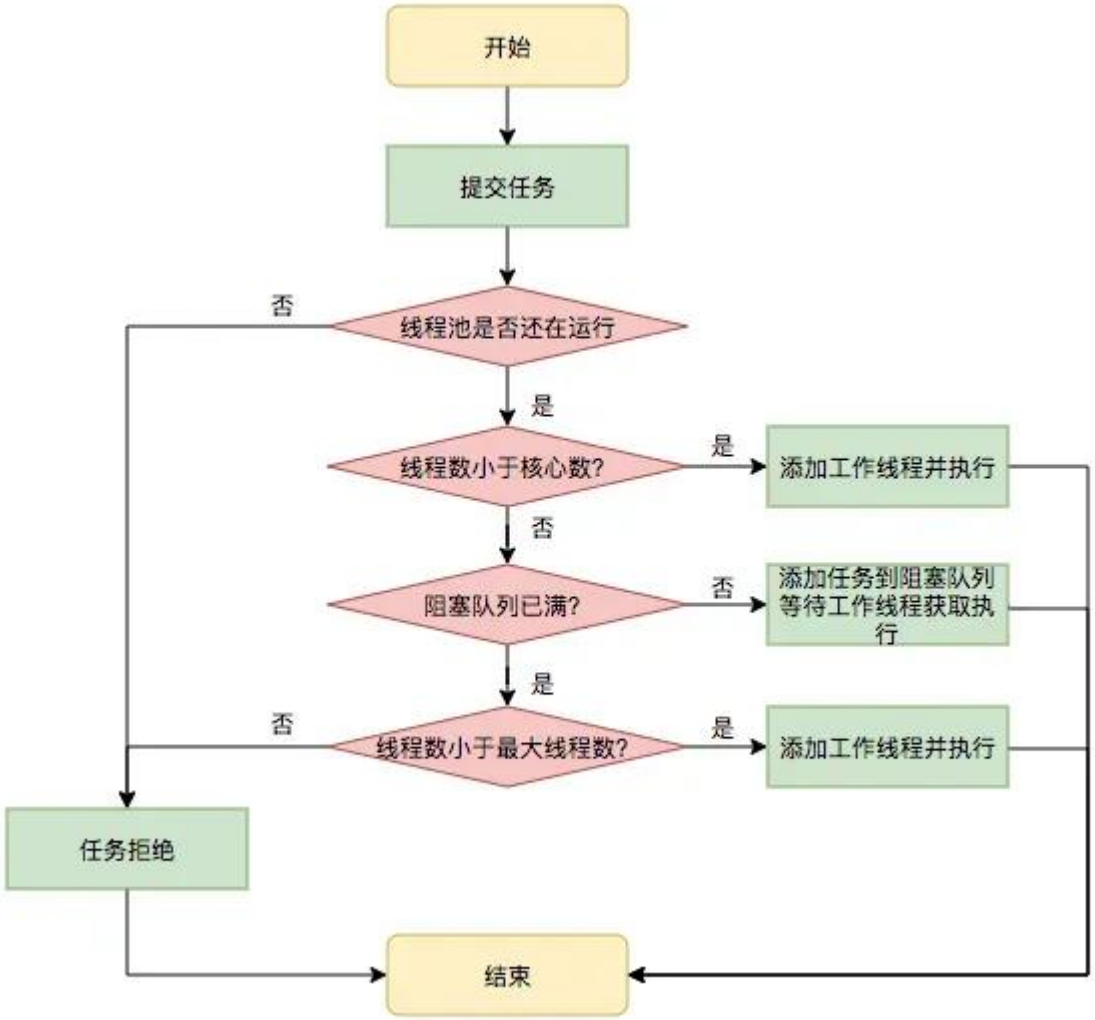
```
// submit 捕获异常
Future<?> future = threadPool.submit(() -> {
    int i = 1 / 0; // 模拟异常
});
try {
    future.get();
} catch (ExecutionException e) {
    System.out.println(任务执行异常 + e.getCause()); // 捕获到算术异常
}
```

还有线程池的使用原则：不能创建后不关闭，否则会导致线程泄露，JVM 无法退出；任务队列的容量要合理设置，太大可能导致内存溢出，太小容易触发拒绝策略；线程数要根据任务类型调整，CPU

密集型任务（比如复杂计算）线程数不宜过多，一般和 CPU 核心数相当，避免线程切换开销，IO 密集型任务可以多设些线程，因为线程大部分时间在等待 IO 完成。

介绍一下线程池的工作原理

线程池是为了减少频繁的创建线程和销毁线程带来的性能损耗，线程池的工作原理如下图：



线程池分为核心线程池，线程池的最大容量，还有等待任务的队列，提交一个任务，如果核心线程没有满，就创建一个线程，如果满了，就是会加入等待队列，如果等待队列满了，就会增加线程，如果达到最大线程数量，如果都达到最大线程数量，就会按照一些丢弃的策略进行处理。

任务执行流程如下：

```
提交任务 → 核心线程是否已满?
  ├─ 未满 → 创建核心线程执行
  └─ 已满 → 任务入队
      ├─ 队列未满 → 等待执行
      └─ 队列已满 → 创建非核心线程
```

```
        ├─ 未达最大线程数 → 执行任务
        └─ 已达最大线程数 → 执行拒绝策略
```

线程池的参数有哪些?

线程池的构造函数有 7 个参数：

```
public ThreadPoolExecutor(int corePoolSize,
                          int maximumPoolSize,
                          long keepAliveTime,
                          TimeUnit unit,
                          BlockingQueue<Runnable> workQueue,
                          ThreadFactory threadFactory,
                          RejectedExecutionHandler handler) {
```

- corePoolSize：线程池核心线程数量。默认情况下，线程池中线程的数量如果 <= corePoolSize，那么即使这些线程处于空闲状态，那也不会被销毁。
- maximumPoolSize：限制了线程池能创建的**最大线程总数**（包括核心线程和非核心线程），当 `corePoolSize` 已满 并且 尝试将新任务加入阻塞队列失败（即队列已满）并且 当前线程数 < `maximumPoolSize`，就会创建新线程执行此任务，但是当 `corePoolSize` 满 并且 队列满 并且 线程数已达 `maximumPoolSize` 并且 又有新任务提交时，就会触发拒绝策略。

```
提交新任务
      |
      v
当前线程数 < corePoolSize?  ---是---> 创建新线程执行此任务
      |
     否
      |    尝试将任务加入工作队列
      |          |
      |<---成功加入? --是---> 任务排队等待执行 (结束，不创建新线程)
      |          |
      |         否 (队列已满)
      |
      v
当前线程数 < maximumPoolSize?  ---是---> 创建新线程执行此任务 (注意：是执行刚提交的这个任务！)
```

```
        |
       否
        |
        v
执行拒绝策略
```

- **keepAliveTime**：当线程池中线程的数量大于 corePoolSize，并且某个线程的空闲时间超过了 keepAliveTime，那么这个线程就会被销毁。
- **unit**：就是 keepAliveTime 时间的单位。
- **workQueue**：工作队列。当没有空闲的线程执行新任务时，该任务就会被放入工作队列中，等待执行。
- **threadFactory**：线程工厂。可以用来给线程取名字等等
- **handler**：拒绝策略。当一个新任务交给线程池，如果此时线程池中有空闲的线程，就会直接执行，如果没有空闲的线程，就会将该任务加入到阻塞队列中，如果阻塞队列满了，就会创建一个新线程，从阻塞队列头部取出一个任务来执行，并将新任务加入到阻塞队列末尾。如果当前线程池中线程的数量等于 maximumPoolSize，就不会创建新线程，就会去执行拒绝策略

线程池工作队列满了有哪些拒接策略?

当线程池的任务队列满了之后，线程池会执行指定的拒绝策略来应对，常用的四种拒绝策略包括：CallerRunsPolicy、AbortPolicy、DiscardPolicy、DiscardOldestPolicy，此外，还可以通过实现 RejectedExecutionHandler 接口来自定义拒绝策略。

四种预置的拒绝策略：

- CallerRunsPolicy，使用线程池的调用者所在的线程去执行被拒绝的任务，除非线程池被停止或者线程池的任务队列已有空缺。
- AbortPolicy，直接抛出一个任务被线程池拒绝的异常。
- DiscardPolicy，不做任何处理，静默拒绝提交的任务。
- DiscardOldestPolicy，抛弃最老的任务，然后执行该任务。
- 自定义拒绝策略，通过实现接口可以自定义任务拒绝策略。

有线程池参数设置的经验吗?

核心线程数（corePoolSize）设置的经验：

- CPU 密集型：corePoolSize = CPU 核数 + 1（避免过多线程竞争 CPU）
- IO 密集型：corePoolSize = CPU 核数 x 2（或更高，具体看 IO 等待时间）

场景一：电商场景，特点瞬时高并发、任务处理时间短，线程池的配置可设置如下：

```
new ThreadPoolExecutor(
    16,                     // corePoolSize = 16（假设 8 核 CPU × 2）
    32,                     // maximumPoolSize = 32（突发流量扩容）
    10, TimeUnit.SECONDS,   // 非核心线程空闲 10 秒回收
    new SynchronousQueue<>(), // 不缓存任务，直接扩容线程
    new AbortPolicy()       // 直接拒绝，避免系统过载
);
```

说明：

- 使用 `SynchronousQueue` 确保任务直达线程，避免队列延迟。
- 拒绝策略快速失败，前端返回“活动火爆”提示，结合降级策略（如缓存预热）。

场景二：后台数据处理服务，特点稳定流量、任务处理时间长（秒级）、允许一定延迟，线程池的配置可设置如下：

```
new ThreadPoolExecutor(
    8,                      // corePoolSize = 8（8 核 CPU）
    8,                      // maximumPoolSize = 8（禁止扩容，避免资源耗尽）
    0, TimeUnit.SECONDS,    // 不回收线程
    new ArrayBlockingQueue<>(1000), // 有界队列，容量 1000
    new CallerRunsPolicy()  // 队列满后由调用线程执行
);
```

说明：

- 固定线程数避免资源波动，队列缓冲任务，拒绝策略兜底。
- 配合监控告警（如队列使用率>80%触发扩容）。

场景三：微服务 HTTP 请求处理，特点 IO 密集型、依赖下游服务响应时间，线程池的配置可设置如下：

```
new ThreadPoolExecutor(
```

```
    16,                     // corePoolSize = 16（8 核 × 2）
    64,                     // maximumPoolSize = 64（应对慢下游）
    60, TimeUnit.SECONDS,   // 非核心线程空闲 60 秒回收
    new LinkedBlockingQueue<>(200), // 有界队列容量 200
    new CustomRetryPolicy() // 自定义拒绝策略（重试或降级）
);
```

说明：

- 根据下游 RT（响应时间）调整线程数，队列防止瞬时峰值。
- 自定义拒绝策略将任务暂存 Redis，异步重试。

核心线程数设置为 0 可不可以?

可以，当核心线程数为 0 的时候，会创建一个非核心线程进行执行。

从下面的源码也可以看到，当核心线程数为 0 时，来了一个任务之后，会先将任务添加到任务队列，同时也会判断当前工作的线程数是否为 0，如果为 0，则会创建线程来执行线程池的任务。

```
int c = ctl.get();
if (workerCountOf(c) < corePoolSize) {
    if (addWorker(command, core: true))
        return;
    c = ctl.get();
}
if (isRunning(c) && workQueue.offer(command)) {
    int recheck = ctl.get();
    if (! isRunning(recheck) && remove(command))
        reject(command);
    else if (workerCountOf(recheck) == 0)
        addWorker( firstTask: null, core: false);
}
else if (!addWorker(command, core: false))
    reject(command);
```

线程池种类有哪些?

- ScheduledThreadPool：可以设置定期的执行任务，它支持定时或周期性执行任务，比如每隔 10 秒钟执行一次任务，我通过这个实现类设置定期执行任务的策略。
- FixedThreadPool：它的核心线程数和最大线程数是一样的，所以可以把它看作是固定线程数的线程池，它的特点是线程池中的线程数除了初始阶段需要从 0 开始增加外，之后的线程数量就是固定的，就算任务数超过线程数，线程池也不会再创建更多的线程来处理任务，而是会把超出线程处理能力的任务放到任务队列中进行等待。而且就算任务队列满了，到了本该继续增加线程数的时候，由于它的最大线程数和核心线程数是一样的，所以也无法再增加新的线程了。
- CachedThreadPool：可以称作可缓存线程池，它的特点在于线程数是几乎可以无限增加的（实际最大可以达到 Integer.MAX_VALUE，为 2^31-1，这个数非常大，所以基本不可能达到），而当线程闲置时还可以对线程进行回收。也就是说该线程池的线程数量不是固定不变的，当然它也有一个用于存储提交任务的队列，但这个队列是 SynchronousQueue，队列的容量为 0，实际不存储任何任务，它只负责对任务进行中转和传递，所以效率比较高。
- SingleThreadExecutor：它会使用唯一的线程去执行任务，原理和 FixedThreadPool 是一样的，只不过这里线程只有一个，如果线程在执行任务的过程中发生异常，线程池也会重新创建一个线程来执行后续的任务。这种线程池由于只有一个线程，所以非常适合用于所有任务都需要按被提交的顺序依次执行的场景，而前几种线程池不一定能够保障任务的执行顺序等于被提交的顺序，因为它们是多线程并行执行的。
- SingleThreadScheduledExecutor：它实际和 ScheduledThreadPool 线程池非常相似，它只是 ScheduledThreadPool 的一个特例，内部只有一个线程。

线程池一般是怎么用的?

Java 中的 Executors 类定义了一些快捷的工具方法，来帮助我们快速创建线程池。《阿里巴巴 Java 开发手册》中提到，禁止使用这些方法来创建线程池，而应该手动 new ThreadPoolExecutor 来创建线程池。这一条规则的背后，是大量血淋淋的生产事故，最典型的就是 newFixedThreadPool 和 newCachedThreadPool，可能因为资源耗尽导致 OOM 问题。

所以，不建议使用 Executors 提供的两种快捷的线程池，原因如下：

- 我们需要根据自己的场景、并发情况来评估线程池的几个核心参数，包括核心线程数、最大线程数、线程回收策略、工作队列的类型，以及拒绝策略，确保线程池的工作行为符合需求，一般都需要设置有界的工作队列和可控的线程数。

- 任何时候，都应该为自定义线程池指定有意义的名称，以方便排查问题。当出现线程数量暴增、线程死锁、线程占用大量 CPU、线程执行出现异常等问题时，我们往往会抓取线程栈。此时，有意义的线程名称，就可以方便我们定位问题。

除了建议手动声明线程池以外，我还建议用一些监控手段来观察线程池的状态。线程池这个组件往往会表现得任劳任怨、默默无闻，除非是出现了拒绝策略，否则压力再大都不会抛出一个异常。如果我们能提前观察到线程池队列的积压，或者线程数量的快速膨胀，往往可以提早发现并解决问题。

线程池和三个线程同时并发比有什么优势?

假设业务中需要处理大量并发任务，直接创建三个线程（或每次任务来了新建线程）存在明显局限，而线程池的优势主要体现在：

- **资源复用，降低开销**：线程创建 / 销毁需要内核态操作（如系统调用），开销较大。线程池会复用核心线程，避免频繁创建线程，尤其在任务量大时，能显著减少资源消耗。而三个固定线程若任务超过负载，要么排队等待（效率低），要么需要临时新建线程（开销高）。
- **控制并发强度，避免资源耗尽**：线程池通过最大线程数和队列限制并发数，防止线程无限制创建（比如瞬间 1000 个任务直接创建 1000 个线程，可能导致 CPU / 内存耗尽）。而三个线程若面对突发大量任务，要么处理不过来（任务堆积），要么手动扩容线程（难以控制）。
- **任务管理更灵活**：线程池支持任务排队、优先级调度、拒绝策略等，能应对任务峰值（如秒杀场景的突发请求）。而三个线程的并发方式，缺乏这些机制，任务处理逻辑会和业务代码耦合（如手动写队列、处理超时）。

比如在接口网关场景中，每秒可能有上百个请求，若用三个线程处理，会导致大量请求排队超时；而线程池通过合理的核心线程数（如 10）、队列（如 200）和最大线程数（如 20），既能高效处理请求，又不会因线程过多拖垮系统。

线程池用了哪些设计模式?

线程池的实现中运用了多种设计模式，这些模式共同支撑了其高效的线程管理和任务调度能力，主要包括：

首先是**工厂模式**，体现在线程池的创建过程中。Java 中 `Executors` 类提供的 `newFixedThreadPool`、`newCachedThreadPool` 等静态方法，本质上是工厂方法，它们封装了 `ThreadPoolExecutor` 复杂的参数配置，根据不同场景返回预配置的线程池实例，简化了用户创建线程池的操作。用户无需关心线程池内部参数的细节，只需根据需求选择对应的工厂方法即可。

其次是**享元模式**，这是线程池的核心设计思想之一。线程池通过复用已创建的线程（尤其是核心线程）来执行多个任务，避免了频繁创建和销毁线程带来的资源消耗。线程作为 “共享资源”，在完成一个任务后不会被销毁，而是回到线程池等待新的任务，这种对线程资源的共享复用，正是享元模式的典型应用，有效提高了系统的资源利用率。

还有**生产者 - 消费者模式**，体现在任务的提交与执行流程中。用户通过 `execute` 或 `submit` 方法提交任务（生产者），任务被放入阻塞队列中；线程池中的工作线程（消费者）不断从队列中取出任务并执行。阻塞队列作为缓冲区，平衡了生产者和消费者的处理速度，避免了任务提交过快导致的线程资源不足，或线程空闲时的资源浪费，实现了任务的异步处理和解耦。

另外，线程池还隐式用到了**模板方法模式**。`ThreadPoolExecutor` 的 `execute` 方法定义了任务执行的整体流程框架：先尝试添加任务到核心线程，失败则加入队列，再失败则创建非核心线程，最后执行拒绝策略。这个流程是固定的模板，而具体的任务执行、线程创建等细节则通过重写 `beforeExecute`、`afterExecute` 等钩子方法进行扩展，用户可以在不改变核心流程的前提下自定义任务执行前后的操作。

线程池中 shutdown ()，shutdownNow()这两个方法有什么作用?

从下面的源码【高亮】注释可以很清晰的看出两者的区别：

- shutdown 使用了以后会置状态为 SHUTDOWN，正在执行的任务会继续执行下去，没有被执行的则中断。此时，则不能再往线程池中添加任何任务，否则将会抛出 RejectedExecutionException 异常
- 而 shutdownNow 为 STOP，并试图停止所有正在执行的线程，不再处理还在池队列中等待的任务，当然，它会返回那些未执行的任务。 它试图终止线程的方法是通过调用 Thread.interrupt() 方法来实现的，但是这种方法的作用有限，如果线程中没有 sleep 、wait、Condition、定时锁等应用, interrupt()方法是无法中断当前的线程的。所以，ShutdownNow()并不代表线程池就一定立即就能退出，它可能必须要等待所有正在执行的任务都执行完成了才能退出。

shutdown 源码：

```
public void shutdown() {
        final ReentrantLock mainLock = this.mainLock;
        mainLock.lock();
        try {
                checkShutdownAccess();
```

```
                // 高亮
                advanceRunState(SHUTDOWN);
                interruptIdleWorkers();
                onShutdown();
        } finally {
                mainLock.unlock();
        }
        tryTerminate();
}
```

shutdownNow 源码：

```
public List<Runnable> shutdownNow() {
        List<Runnable> tasks;
        final ReentrantLock mainLock = this.mainLock;
        mainLock.lock();
        try {
                checkShutdownAccess();
                // 高亮
                advanceRunState(STOP);
                interruptWorkers();
                // 高亮
                tasks = drainQueue();
        } finally {
                mainLock.unlock();
        }
        tryTerminate();
        // 高亮
        return tasks;
}
```

提交给线程池中的任务可以被撤回吗?

可以，当向线程池提交任务时，会得到一个 `Future` 对象。这个 `Future` 对象提供了几种方法来管理任务的执行，包括取消任务。

取消任务的主要方法是 `Future` 接口中的 `cancel(boolean mayInterruptIfRunning)`方法。这个方法尝试取消执行的任务。参数 `mayInterruptIfRunning` 指示是否允许中断正在执行的任务。如果设置为 `true`，则表示如果任务已经开始执行，那么允许中断任务；如果设置为 `false`，任务已经开始执行则不会被中断。

```
public interface Future<V> {
    // 是否取消线程的执行
    boolean cancel(boolean mayInterruptIfRunning);
    // 线程是否被取消
    boolean isCancelled();
    //线程是否执行完毕
    boolean isDone();
      // 立即获得线程返回的结果
    V get() throws InterruptedException, ExecutionException;
      // 延时时间后再获得线程返回的结果
    V get(long timeout, TimeUnit unit)
        throws InterruptedException, ExecutionException, TimeoutException;
}
```

取消线程池中任务的方式，代码如下，通过 future 对象的 cancel(boolean) 函数来定向取消特定的任务。

```
public static void main(String[] args) {
        ExecutorService service = Executors.newSingleThreadExecutor();
        Future future = service.submit(new TheradDemo());

        try {
          // 可能抛出异常
            future.get();
        } catch (InterruptedException e) {
            e.printStackTrace();
        } catch (ExecutionException e) {
            e.printStackTrace();
        }finally {
          //终止任务的执行
```

```
            future.cancel(true);
        }
 }
```

场景

多线程打印奇偶数，怎么控制打印的顺序

可以利用 wait()和 notify()来控制线程的执行顺序。

以下是一个基于这种方法的简单示例：

```
public class PrintOddEven {
    private static final Object lock = new Object();
    private static int count = 1;
    private static final int MAX_COUNT = 10;

    public static void main(String[] args) {
        Runnable printOdd = () -> {
            synchronized (lock) {
                while (count <= MAX_COUNT) {
                    if (count % 2 != 0) {
                        System.out.println(Thread.currentThread().getName() + ": " + count++);
                        lock.notify();
                    } else {
                        try {
                            lock.wait();
                        } catch (InterruptedException e) {
                            e.printStackTrace();
                        }
                    }
                }
            }
        };

        Runnable printEven = () -> {
            synchronized (lock) {
```

```
                while (count <= MAX_COUNT) {
                    if (count % 2 == 0) {
                        System.out.println(Thread.currentThread().getName() + ": " + count++);
                        lock.notify();
                    } else {
                        try {
                            lock.wait();
                        } catch (InterruptedException e) {
                            e.printStackTrace();
                        }
                    }
                }
            }
        };

        Thread oddThread = new Thread(printOdd, "OddThread");
        Thread evenThread = new Thread(printEven, "EvenThread");

        oddThread.start();
        evenThread.start();
    }
}
```

在上面的示例中，通过一个共享的锁对象 lock 来控制两个线程的交替执行。一个线程负责打印奇数，另一个线程负责打印偶数，通过 wait()和 notify()方法来在两个线程之间实现顺序控制。当当前应该打印奇数时，偶数线程会进入等待状态，反之亦然。

单例模型既然已经用了 synchronized，为什么还要在加 volatile?

使用 `synchronized` 和 `volatile` 一起，可以创建一个既线程安全又能正确初始化的单例模式，避免了多线程环境下的各种潜在问题。这是一种比较完善的线程安全的单例模式实现方式，尤其适用于高并发环境。

```
public class Singleton {
    private static volatile Singleton instance;

    private Singleton() {}
```

```
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

`synchronized` 关键字的作用用于确保在多线程环境下，只有一个线程能够进入同步块（这里是 `synchronized (Singleton.class)`）。在创建单例对象时，通过 `synchronized` 保证了创建过程的线程安全性，避免多个线程同时创建多个单例对象。

`volatile` 确保了对象引用的可见性和创建过程的有序性，避免了由于指令重排序而导致的错误。

`instance = new Singleton();` 这行代码并不是一个原子操作，它实际上可以分解为以下几个步骤：

- 分配内存空间。
- 实例化对象。
- 将对象引用赋值给 `instance`。

由于 Java 内存模型允许编译器和处理器对指令进行重排序，在没有 `volatile` 的情况下，可能会出现重排序，例如先将对象引用赋值给 `instance`，但对象的实例化操作尚未完成。

这样，其他线程在检查 `instance == null` 时，会认为单例已经创建，从而得到一个未完全初始化的对象，导致错误。

`volatile` 可以保证变量的可见性和禁止指令重排序。它确保对 `instance` 的修改对所有线程都是可见的，并且保证了上述三个步骤按顺序执行，避免了在单例创建过程中因指令重排序而导致的问题。

3 个线程并发执行，1 个线程等待这三个线程全部执行完在执行，怎么实现?

可以使用 `CountDownLatch` 来实现 3 个线程并发执行，另一个线程等待这三个线程全部执行完再执行的需求。以下是具体的实现步骤：

- 创建一个 `CountDownLatch` 对象，并将计数器初始化为 3，因为有 3 个线程需要等待。
- 创建 3 个并发执行的线程，在每个线程的任务结束时调用 `countDown` 方法将计数器减 1。
- 创建第 4 个线程，使用 `await` 方法等待计数器为 0，即等待其他 3 个线程完成任务。

```
import java.util.concurrent.CountDownLatch;


public class CountDownLatchExample {
    public static void main(String[] args) {
        // 创建一个 CountDownLatch，初始计数为 3
        CountDownLatch latch = new CountDownLatch(3);


        // 创建并启动 3 个并发线程
        for (int i = 0; i < 3; i++) {
            final int threadNumber = i + 1;
            new Thread(() -> {
                try {
                    System.out.println("Thread " + threadNumber + " is working.");
                    // 模拟线程执行任务
                    Thread.sleep((long) (Math.random() * 1000));
                    System.out.println("Thread " + threadNumber + " has finished.");
                } catch (InterruptedException e) {
                    e.printStackTrace();
                } finally {
                    // 任务完成后，计数器减 1
                    latch.countDown();
                }
            }).start();
        }


        // 创建并启动第 4 个线程，等待其他 3 个线程完成
        new Thread(() -> {
            try {
                System.out.println("Waiting for other threads to finish.");
                // 等待计数器为 0
                latch.await();
```

```
                System.out.println("All threads have finished, this thread starts to work.");
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }).start();
    }
}
```

代码解释：

- 首先，创建了一个 `CountDownLatch` 对象 `latch`，并将其初始计数设置为 3。
- 然后，使用 `for` 循环创建并启动 3 个线程。每个线程会执行一些工作（这里使用 `Thread.sleep` 模拟），在工作完成后，会调用 `latch.countDown()` 方法，将 `latch` 的计数减 1。
- 最后，创建第 4 个线程。这个线程在开始时调用 `latch.await()` 方法，它会阻塞，直到 `latch` 的计数为 0，即前面 3 个线程都调用了 `countDown()` 方法。一旦计数为 0，该线程将继续执行后续任务。

假设两个线程并发读写同一个整型变量，初始值为零，每个线程加 50 次，结果可能是什么?

在没有任何同步机制的情况下，两个线程并发对同一个整型变量进行 50 次加 1 操作，最终结果可能是 100，也可能小于 100，最坏的结果是 50，也就是最终的结果可能是在［50, 100］。

小于 100 情况的分析，由于对整型变量的 `num++` 操作不是原子操作，它实际上包含了三个步骤：读取变量的值、将值加 1、将新值写回变量。在多线程环境下，可能会出现线程安全问题。例如，线程 1 和线程 2 同时读取了变量的当前值，然后各自将其加 1，最后都将相同的新值写回变量，这就导致了一次加 1 操作的丢失。这种情况会多次发生，最终结果就会小于 100。

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicIntegerAddition {
    private static AtomicInteger num = new AtomicInteger(0);

    public static void main(String[] args) throws InterruptedException {
        Thread thread1 = new Thread(() -> {
            for (int i = 0; i < 50; i++) {
```

```
                num.incrementAndGet();
            }
        });

        Thread thread2 = new Thread(() -> {
            for (int i = 0; i < 50; i++) {
                num.incrementAndGet();
            }
        });

        thread1.start();
        thread2.start();

        thread1.join();
        thread2.join();

        System.out.println("最终结果: " + num.get());
    }
}
```

第二种方式：通过 `synchronized` 关键字或 `ReentrantLock` 确保操作的互斥性，代码如下：

```
public class SynchronizedAddition {
    private static int num = 0;
    private static final Object lock = new Object();

    public static void main(String[] args) throws InterruptedException {
        Thread thread1 = new Thread(() -> {
            for (int i = 0; i < 50; i++) {
                synchronized (lock) {
                    num++;
                }
            }
        });

        Thread thread2 = new Thread(() -> {
```

```
            for (int i = 0; i < 50; i++) {
                synchronized (lock) {
                    num++;
                }
            }
        });

        thread1.start();
        thread2.start();

        thread1.join();
        thread2.join();

        System.out.println("最终结果: " + num);
    }
}
```
